Machine Learning

Chapitre 2 : Classification

Mme BEDAD Fatima
fatima.bedad@univ-temouchent.edu.dz

2023-2024

Table of Contents

1) Dataset Loading
2) Data Preprocessing + Exploratory Data Analysis
3) Feature selection
4) Dimensionality Reduction
5) Training and Test Sets
6) Supervised learning
7) Unsupervised learning
8) Save and Load Machine Learning Models

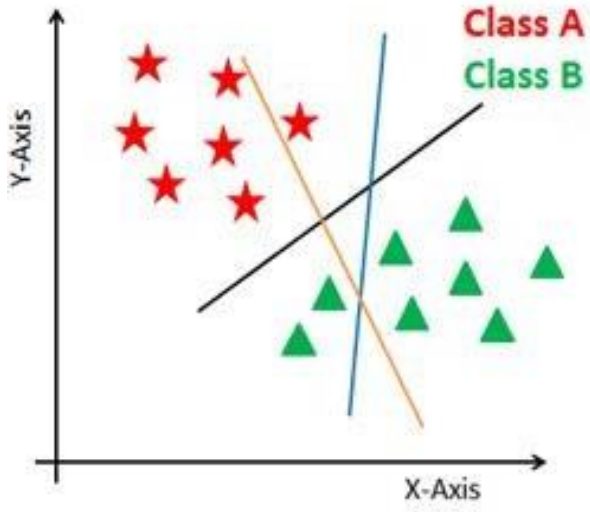
Classification

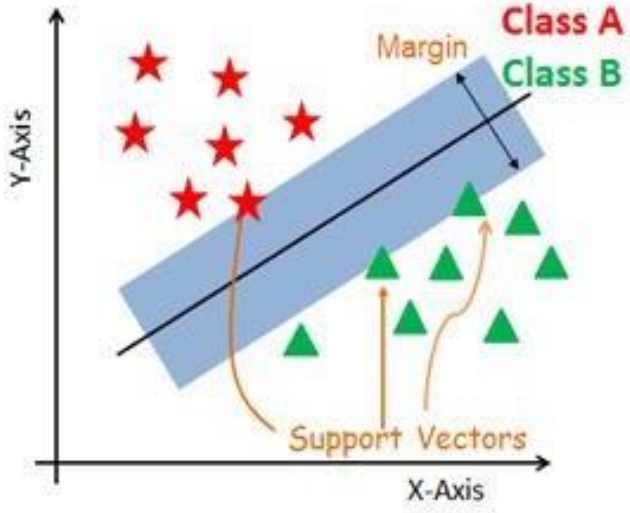
2. Classification: Support Vector Machines

The objective is to **select a hyperplane** with the **maximum possible margin** between support vectors in the given dataset. **SVM** searches for the maximum marginal hyperplane in the following steps:

1) **Generate hyperplanes** that separate the classes in the best way. Left-hand side figure showing three hyperplanes: black, blue and orange. Here, the blue and orange have higher classification errors, but **the black** is separating the two classes correctly.

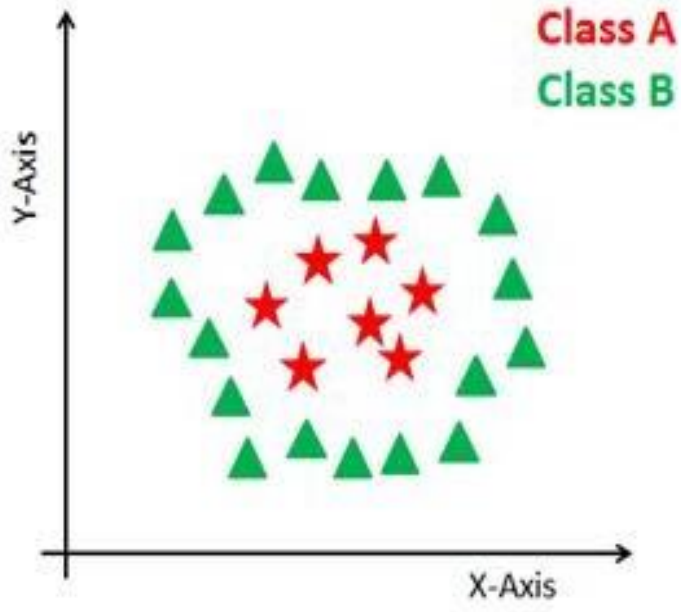
2) **Select the right hyperplane** with the maximum separation from either nearest data points as shown in the right-hand side figure.

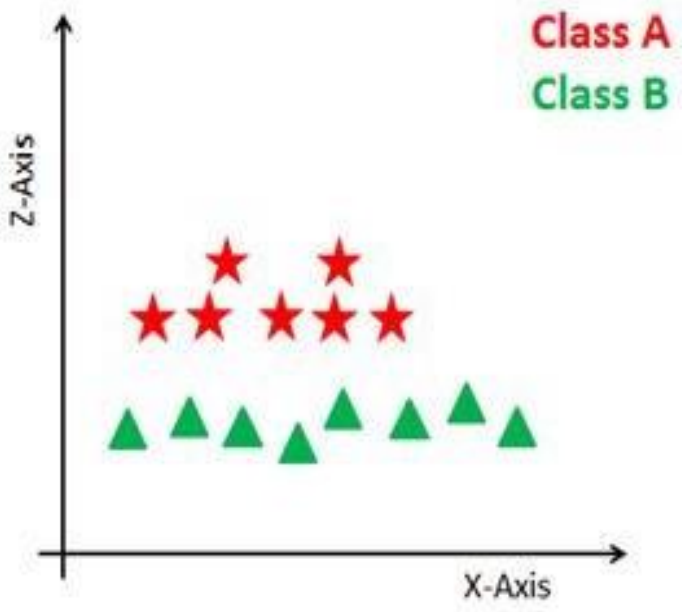




2. Classification: Support Vector Machines - Kernel

• Some problems **can not be solved using linear hyperplane**, as shown in the figure below (left-hand side).
• In such situation, **SVM uses a kernel** to transform the input space to a higher dimensional space as shown on the right (**z=x^2 + y^2**).



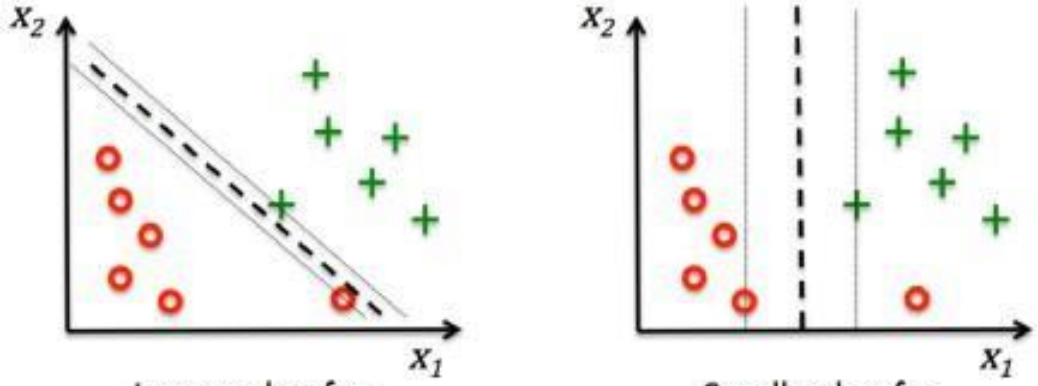


2. Classification: Support Vector Machines - Kernel + Tuning Hyperparameters

Regularization: the **C parameter** controls how much you want to **punish your model** for each misclassified point for a given curve:
Large values of C (Large effect of noisy points. A plane with very few misclassifications will be given precedence). Small Values of C (Low effect of noisy points. Planes that separate the points well will be found, even if there are some misclassifications).

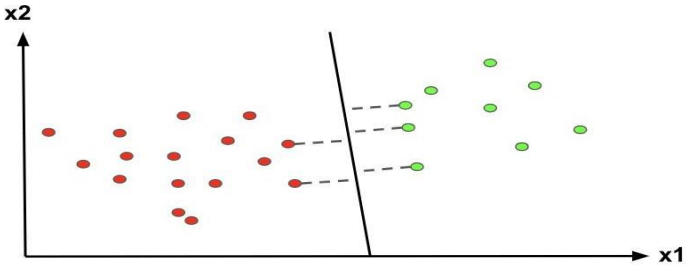
Gamma (Kernel coefficient for rbf, poly and sigmoid): the **high value** of gamma considers only **nearby points** in calculating the separation line, while a **low value** of gamma considers **all the data points** in the calculation of the separation line.

2. Classification: Support Vector Machines - Kernel + Tuning Hyperparameters



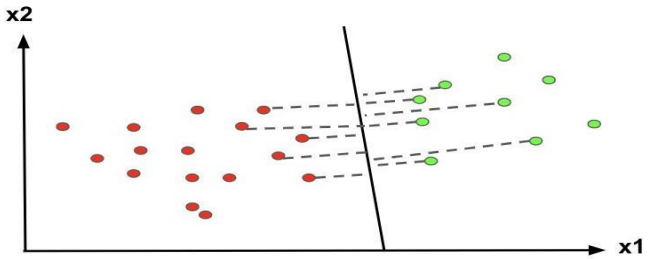
Large value for parameter C

Small value for parameter C



High Gamma

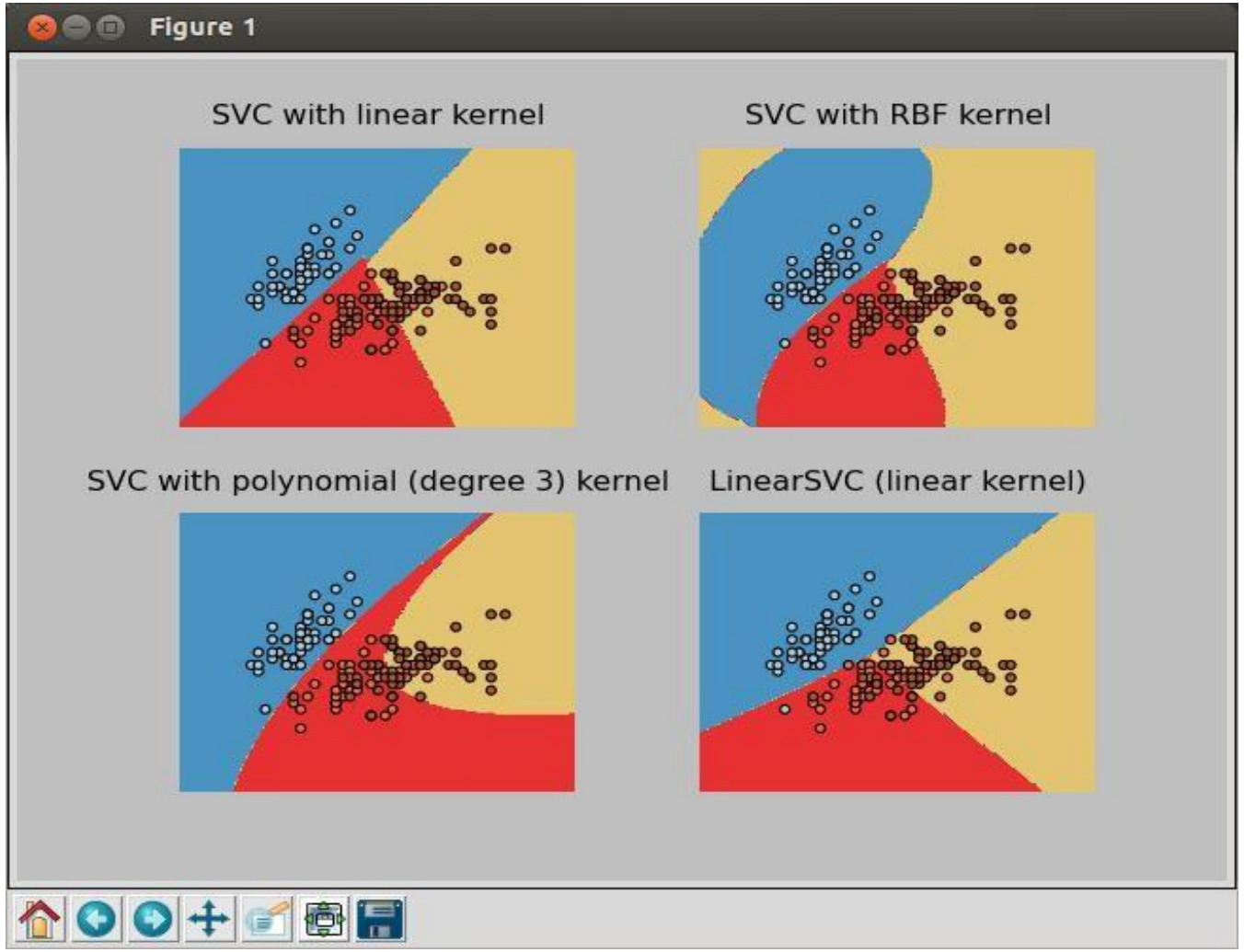
- only near points are considered.



Low Gamma

- far away points are also considered

2. Classification: Support Vector Machines - Kernel + Tuning Hyperparameters



2. Classification: Support Vector Machines -Kernel + Tuning Hyperparameters

```
from sklearn.svm import SVC
# linear kernel
svc_cls = SVC(kernel='linear')
# Polynomial Kernel
svc_cls = SVC(kernel='poly', degree=5, C=10, gamma=0.1)
# Gaussian Kernel
svc_cls = SVC(kernel='rbf', C=10, gamma=0.1)
# Sigmoid Kernel
svc_cls = SVC(kernel='sigmoid', C=10, gamma=0.1)

svc_cls.fit(X_train, y_train)
```

Classification Algorithms :

Data prepocessing
Duplicate Rows and Missing Values
Correlation and Data visualisation
Handling Outlier
Mapping of Data feacture values
Selecting Relevent feacture

Data Prepocessing

```
# Import Python Libraries
import numpy as np
import scipy as sp
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("/content/breast_cancer.csv")

# Display dimensions of dataframe
print(df.shape)
print(df.info())

print("-----------------------------------------------------------------")
# Display first 10 records
print(df.head(10))
```

```
print("-----------------------------------------------------------------")
# List the column names
print(df.columns)
```

```
(569, 33)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
None
```

```
--------------------------------------------------------------------------------
         id diagnosis  ...  fractal_dimension_worst  Unnamed: 32
0    842302         M  ...                   0.11890          NaN
1    842517         M  ...                   0.08902          NaN
2  84300903         M  ...                   0.08758          NaN
3  84348301         M  ...                   0.17300          NaN
4  84358402         M  ...                   0.07678          NaN
5    843786         M  ...                   0.12440          NaN
6    844359         M  ...                   0.08368          NaN
7  84458202         M  ...                   0.11510          NaN
8    844981         M  ...                   0.10720          NaN
9  84501001         M  ...                   0.20750          NaN

[10 rows x 33 columns]
```

```python
# Remove the last column .i.e the 33rd one as it is not needed + 0

# df.drop(columns=["B", "C"])
# df.drop(["B", "C"], axis=1)
# df.drop([0, 1]) # Drop a row by index

df.drop(df.columns[[-1, 0]], axis=1, inplace=True)
df.info()

print("----------------------------------------------------------------------")
# Display statistics for numeric columns
print(df.describe())
```

```python
# Lets know how many values for malignant and for benign type of cancer
print ("Total number of diagnosis are ", df.shape[0], ", ",
       df.diagnosis.value_counts()["B"], "Benign and Malignant are",
       df.diagnosis.value_counts()["M"])
print("\n")
print("diagnosis        : \n",df["diagnosis"].value_counts())
print("\n")
print(df["diagnosis"].value_counts(normalize=True))

# plot.bar(title='') function is used to plot bargraph
df["diagnosis"].value_counts().plot.bar(title="diagnosis")
```

Duplicate Rows and Missing Values

```
# Dropping the duplicate rows
duplicate_rows = df[df.duplicated()]
print("number of duplicate rows : ",duplicate_rows.shape)

# df.drop_duplicates(inplace=True)
```

```
# To check missing values
print("SUM : \n",df.isnull().sum())
sns.heatmap(df.isnull(), cbar=False, yticklabels=False, cmap="viridis")

# df["col"] = df["col"].fillna(df["col"].mean())
# df["col"] = df["col"].fillna(df["col"].interpolate())
```

Correlation and Data visualisation

```
# Find the pairwise correlation of all columns in the dataframe.
print(df.corr())

# Heatmap
print("\n")
corrMatrix = df.corr()
fig, ax = plt.subplots(figsize=(16,16))
#sns.heatmap(corrMatrix, annot=True)
sns.heatmap(corrMatrix, annot=True, linewidth=0.01, square=True, cmap="RdBu", linecolor="black")
```

Handling Outlier

```
# Handling Outlier
fig, ax = plt.subplots(figsize=(50,16))
sns.boxplot(data=df, orient="v", palette="Set2")
plt.show
```

```
# One column
sns.boxplot(x=df["perimeter_mean"], data=df)
sns.swarmplot(x=df["perimeter_mean"],  data=df, color=".25")
```

```python
# Remove Outlier

def remove_outlier(col):
    sorted(col)
    Q1, Q3 = col.quantile([0.25, 0.75])
    IQR = Q3 - Q1
    print("Q1 = ", Q1, " Q3 = ", Q3, " IQR = ", IQR)
    lower_range = Q1 - (1.5 * IQR)
    upper_range = Q3 + (1.5 * IQR)
    return lower_range, upper_range

# Extract all column names as list in Pandas Dataframe
lst_all_cols = df.columns.tolist()
print("lst_all_cols : \n",lst_all_cols)

print("\n")
```

```python
# Numerical data distribution
print(list(set(df.dtypes.tolist())))
df_num = df.select_dtypes(include = ["float64"])
lst_num_cols = df_num.columns.tolist()
print("Numerical Data : \n",lst_num_cols)
print("\n")
```

```python
df_copy = df.copy()

indx = 0
for col in lst_num_cols:
    print(indx)
    lower_range, upper_range =  remove_outlier(df_copy[col])
    df_copy[col] = np.where(df_copy[col] < lower_range, lower_range, df_copy[col])
    df_copy[col] = np.where(df_copy[col] > upper_range, upper_range, df_copy[col])
    indx = indx +1
    print("------------------------------")
```

```
fig, ax = plt.subplots(figsize=(50,16))
sns.boxplot(data=df_copy, orient="v", palette="Set2")
plt.show
```

Mapping of Data feacture values

```
# Mapping of data feature values

# df_copy
df_copy["diagnosis"] = df_copy["diagnosis"].map({"B":0,"M":1})
print(df_copy["diagnosis"].head(30))
```

Selecting Relevent feacture

```
corrMatrix_1 = df_copy.corr()
# corrMatrix_1 = df.corr()

fig, ax = plt.subplots(figsize=(25,20))
#sns.heatmap(corrMatrix_1, annot=True)
sns.heatmap(corrMatrix_1, annot=True, linewidth=0.01, square=True, cmap="RdBu", lin
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	fractal_dimension_mean	radius_se	texture_se
diagnosis	1	0.75	0.42	0.76	0.75	0.37	0.61	0.73	0.79	0.33	-0.0068	0.64	-0.00015
radius_mean	0.75	1	0.33	1	0.99	0.17	0.5	0.67	0.81	0.15	-0.32	0.68	-0.11
texture_mean	0.42	0.33	1	0.34	0.34	-0.016	0.24	0.32	0.3	0.08	-0.066	0.33	0.41
perimeter_mean	0.76	1	0.34	1	0.99	0.2	0.55	0.71	0.84	0.18	-0.27	0.7	-0.1
area_mean	0.75	0.99	0.34	0.99	1	0.17	0.5	0.68	0.81	0.16	-0.3	0.71	-0.087
smoothness_mean	0.37	0.17	-0.016	0.2	0.17	1	0.66	0.53	0.56	0.56	0.59	0.32	0.096
compactness_mean	0.61	0.5	0.24	0.55	0.5	0.66	1	0.89	0.83	0.59	0.56	0.53	0.045
concavity_mean	0.73	0.67	0.32	0.71	0.68	0.53	0.89	1	0.93	0.49	0.33	0.64	0.069
concave points_mean	0.79	0.81	0.3	0.84	0.81	0.56	0.83	0.93	1	0.46	0.17	0.73	0.024
symmetry_mean	0.33	0.15	0.08	0.18	0.16	0.56	0.59	0.49	0.46	1	0.47	0.34	0.15
fractal_dimension_mean	-0.0068	-0.32	-0.066	-0.27	-0.3	0.59	0.56	0.33	0.17	0.47	1	0.0062	0.17
radius_se	0.64	0.68	0.33	0.7	0.71	0.32	0.53	0.64	0.73	0.34	0.0062	1	0.26
texture_se	-0.00015	-0.11	0.41	-0.1	-0.087	0.096	0.045	0.069	0.024	0.15	0.17	0.26	1

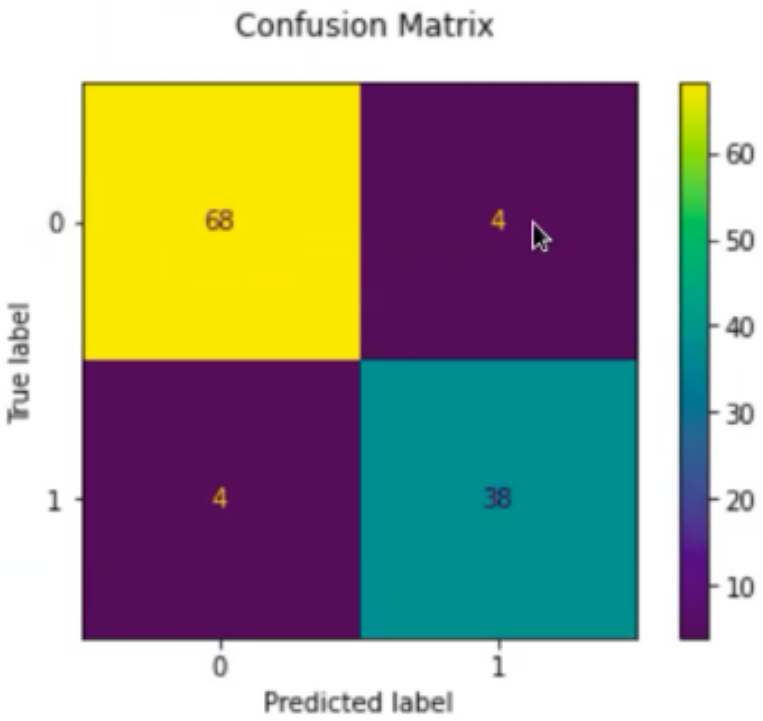
```python
# Correlation with output variable
cor_target = abs(corrMatrix_1["diagnosis"])

# Selecting highly correlated features : 0.5
relevant_features = cor_target[cor_target>0.5]
print("relevant_features : ",relevant_features.shape,"\n",relevant_features)

print("-----------------------------------------------------------------------")
lst_columns = relevant_features.index.to_list()

my_data = pd.DataFrame(df_copy, columns= lst_columns)
print (my_data.head(10))

print("-----------------------------------------------------------------------")
my_cols = my_data.columns.to_list()
print("List the column names : ",len(my_cols),"\n",my_cols)
```

```python
# Define X variables and our target(y)
"""
features = list(my_data.columns[1:16])

X = my_data.loc[:,features]
y = my_data.loc[:, "diagnosis"]
print("X : ",X.shape," y : ",y.shape)
"""

X = my_data.drop(["diagnosis"],axis=1).values
y = my_data["diagnosis"].values
print("X : ",X.shape," y : ",y.shape)

# Split into Input and Output Elements
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,
                                    y, test_size= 0.20, random_state=100, stratify=y)

print("X_train = ",X_train.shape ," y_train = ", y_train.shape)
print("X_test  = ",X_test.shape ," y_test = ", y_test.shape)
```

Support vector machine SVM

```
from sklearn import datasets, svm, metrics

# Create a classifier: a support vector classifier
# classifier = svm.SVC()
classifier = svm.SVC(kernel="linear")
# classifier = svm.SVC(kernel="poly", degree=5, C=10, gamma=0.01)
# classifier = svm.SVC(kernel="sigmoid", C=10, gamma=0.001)
# classifier = svm.SVC(kernel="rbf", C=10, gamma=0.01)

# Train the classifier
classifier.fit(X_train, y_train)

# Now predict the value of X_test
predicted = classifier.predict(X_test)
```

```python
# Classification report
print("Classification report : \n", classifier,"\n",
      metrics.classification_report(y_test, predicted))

disp = metrics.plot_confusion_matrix(classifier, X_test, y_test)
disp.figure_.suptitle("Confusion Matrix")

print("Confusion matrix: \n", disp.confusion_matrix)
```



2. Classification: Support Vector Machines -Kernel + Tuning Hyperparameters

```
import matplotlib.pyplot as plt
from sklearn import datasets, svm, metrics
from sklearn.model_selection import train_test_split

# Load the dataset
digits = datasets.load_digits()
print("digits shape : ",digits.images.shape)

fig = plt.figure()
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.tight_layout()
    plt.imshow(digits.images[i], cmap="gray",
interpolation="none")
    plt.title("Digit: {}".format(digits.target[i]))
    plt.xticks([])
    plt.yticks([])
fig

# To apply a classifier on this data, we need to flatten the image,
# to turn the data in a (samples, feature) matrix:
n_samples = len(digits.images)
data = digits.images.reshape((n_samples, -1))
print("data shape : ",data.shape)
```

```
# Split data into train and test subsets
X_train, X_test, y_train, y_test = train_test_split(
    data, digits.target, test_size=0.2, shuffle=False)
print("X_train : ",X_train.shape," y_train : ",y_train.shape)
print("X_test  : ",X_test.shape," y_test : ",y_test.shape)

print("----------------------------------------------------------------------")
# Create a classifier: a support vector classifier
classifier = svm.SVC(kernel="linear")
# classifier = svm.SVC(kernel="poly", degree=5, C=10, gamma=0.01)
# classifier = svm.SVC(kernel="sigmoid", C=10, gamma=0.001)
# classifier = svm.SVC(kernel="rbf", C=10, gamma=0.01)


classifier.fit(X_train, y_train)

# Now predict the value of the digit on the second half:
predicted = classifier.predict(X_test)


print("Classification report : \n", classifier,"\n",
metrics.classification_report(y_test,  predicted))
disp = metrics.plot_confusion_matrix(classifier, X_test, y_test)
disp.figure_.suptitle("Confusion  Matrix")

print("Confusion matrix: \n", disp.confusion_matrix)
```

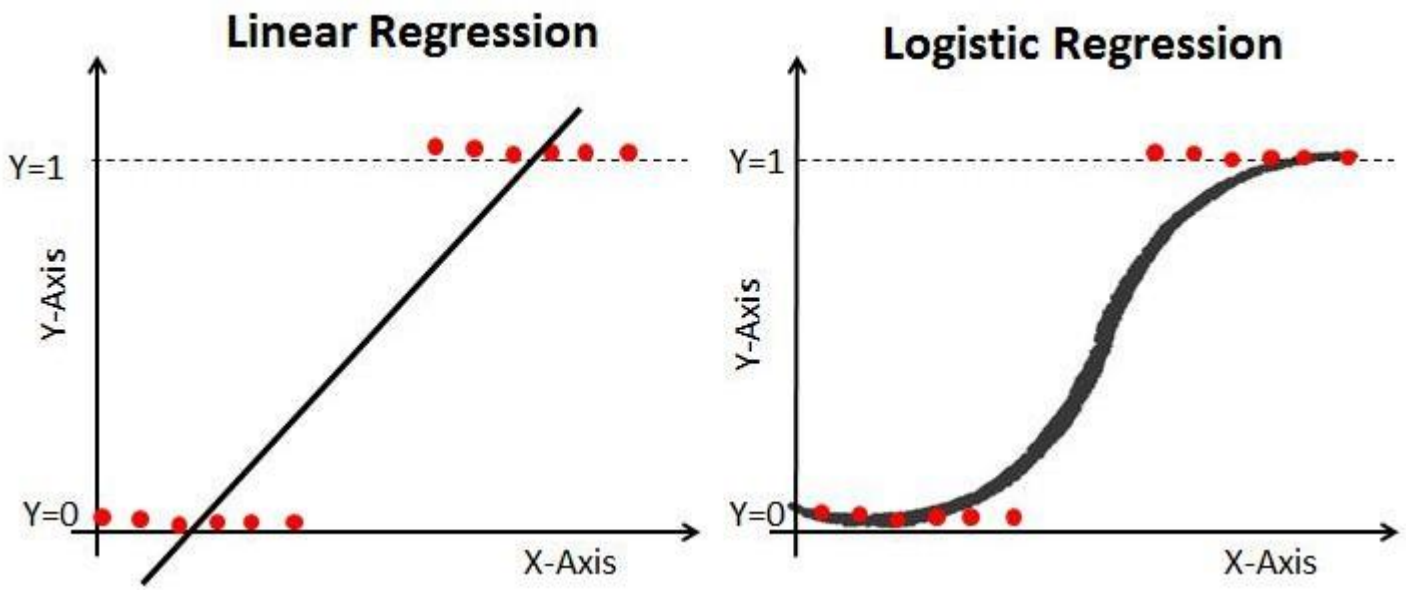
2. Classification: Logistic Regression Classifier

Logistic regression classifier: is a fundamental classification technique. It belongs to the group of linear classifiers and is somewhat similar to **polynomial and linear regression**. Logistic regression is **fast and relatively uncomplicated**, and it's **convenient** for you to interpret the results. Although it's essentially a method for **binary classification**, it can also be applied to **multiclass problems**.

Your goal is to find the logistic regression function $p(x)$ such that the predicted responses $p(x_i)$ are as close as possible to the actual response y_i for each observation $i = 1, \ldots, n$.

Sigmoid Function :

$$f(x) = \frac{1}{1 + e^{-(x)}}$$



2. Classification: Logistic Regression Classifier

```
from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

dat = datasets.load_breast_cancer()
print("Examples = ",dat.data.shape ," Labels = ", dat.target.shape)
X_train, X_test, Y_train, y_test = train_test_split(dat.data, dat.target, test_size= 0.20, random_state=100)
# default = 'auto'
logreg = LogisticRegression()
logreg.fit(X_train,Y_train)
predicted = logreg.predict(X_test)

print("Classification report for classifier : \n", metrics.classification_report(y_test, predicted))

disp = metrics.plot_confusion_matrix(logreg, X_test, y_test)

disp.figure_.suptitle("Confusion Matrix")
print("Confusion matrix: \n", disp.confusion_matrix)
plt.show()
```

2. Classification: Logistic Regression Classifier

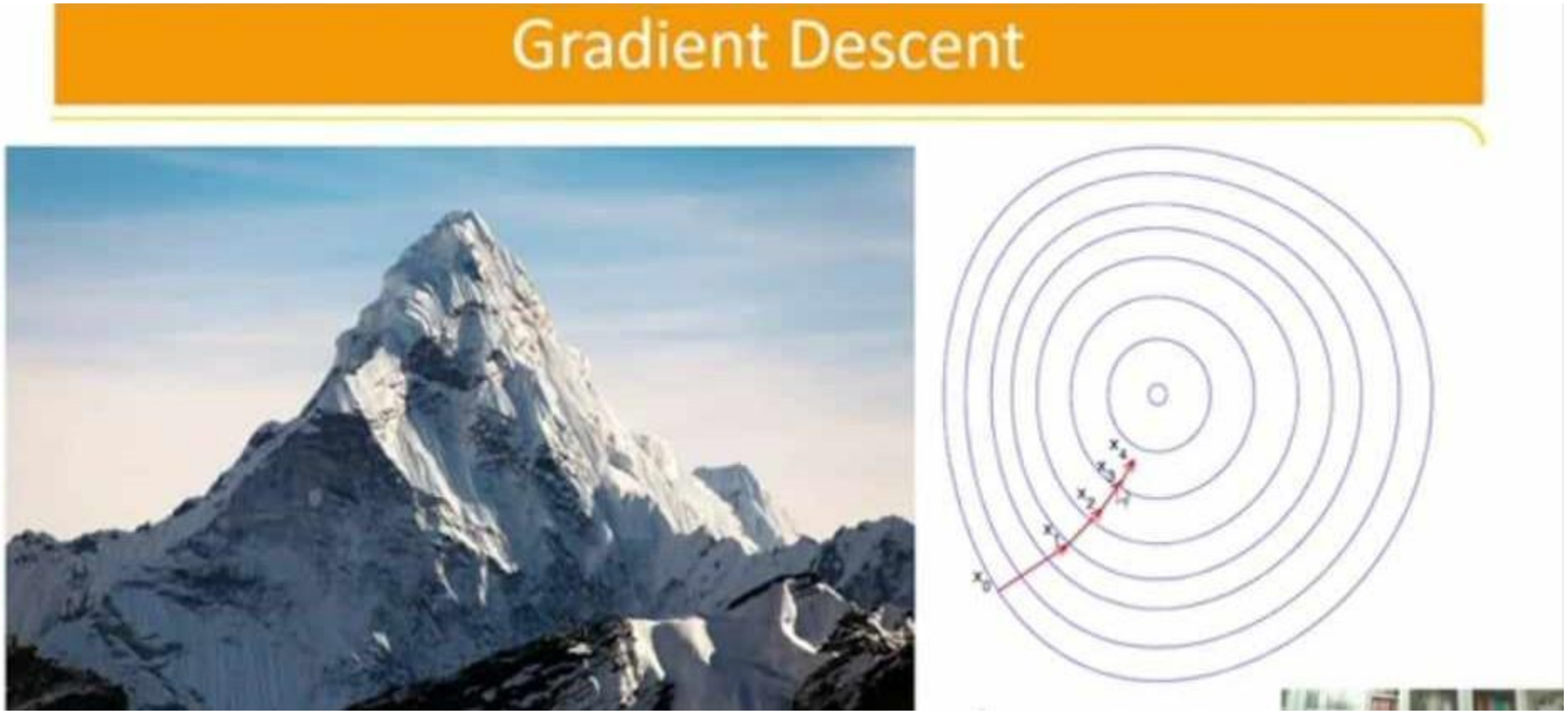
Advantages:

Because of its efficient and straightforward nature, doesn't require high computation power, easy to implement, easily interpretable, used widely by data analyst and scientist. Also, it doesn't require scaling of features. Logistic regression provides a probability score for observations.

Disadvantages:

Logistic regression is not able to handle a large number of categorical features/variables. It is vulnerable to overfitting. Also, can't solve the non-linear problem with the logistic regression that is why it requires a transformation of non-linear features. Logistic regression will not perform well with independent variables that are not correlated to the target variable and are very similar or correlated to each other.

2. Classification: Stochastic Gradient Descent



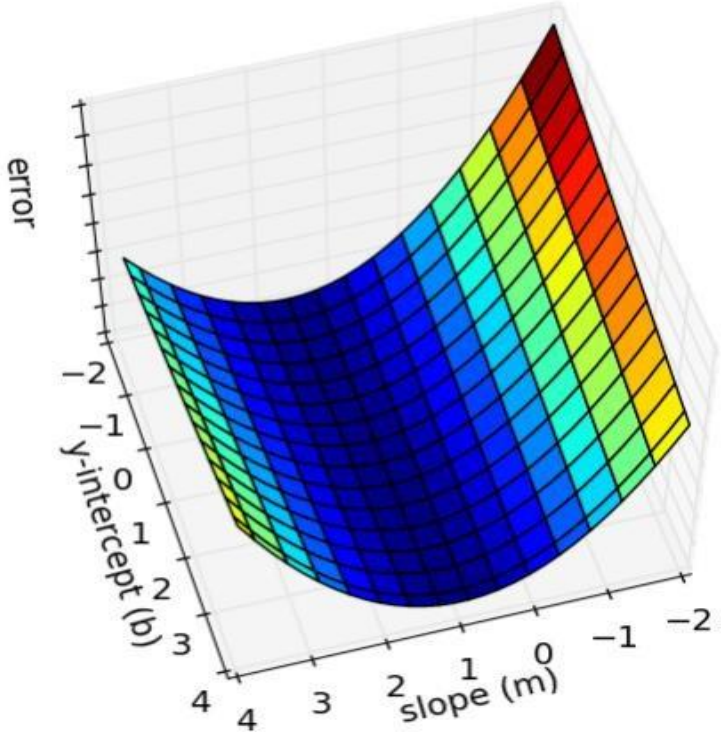
2. Classification: Stochastic Gradient Descent

Gradient Descent : at a theoretical level, gradient descent is an algorithm that minimizes functions. Given a function defined by a set of parameters, gradient descent starts with an initial set of parameter values and iteratively moves toward a set of parameter values that minimize the function. This iterative minimization is achieved using calculus, taking steps in the negative direction of the function gradient.

Example: y = mx + b line equation where **m** is the line's slope and **b** is the line's y-intercept. To find the best line for our data, we need to find the best set of slope **m** and y-intercept **b** values.

f(m,b): error function (also called a cost function) that measures how "good" a given line is.

Each point in this two-dimensional space represents a line. The height of the function at each point is the error value for that line.



2. Classification: Stochastic Gradient Descent

Stochastic gradient descent (SGD): also known as incremental gradient descent, is an iterative method for optimizing a differentiable objective function, a stochastic approximation of gradient descent optimization.

SGD has been successfully applied to large-scale and sparse **machine learning problems** often encountered in **text classification** and **natural language processing**. Given that the data is **sparse**, the classifiers in this module **easily scale** to problems with more than **10^5 training examples** and more than **10^5 features**.

2. Classification: Stochastic Gradient Descent

```
from sklearn.linear_model import SGDClassifier

X = [[0., 0.], [1., 1.]]
y = [0, 1]

# loss : hinge, log, modified_huber, squared_hinge, perceptron
clf = SGDClassifier(loss="hinge", penalty="l2", max_iter=5)
clf.fit(X, y)


predicted = clf.predict(X_test)
```

2. Classification: Stochastic Gradient Descent

```
from sklearn.linear_model import SGDClassifier
from sklearn import metrics

# Create the classifier
sgdc_cls = SGDClassifier(loss="log", alpha=0.01, penalty="l2", random_state =42)

# Train the classifier
sgdc_cls.fit(X_train,y_train)

# Predict the value of X_test
predicted = sgdc_cls.predict(X_test)

# Classification report
print("Classifier : ", sgdc_cls)
print("Classification report for classifier : \n", metrics.classification_report(y_test, predicted))

disp = metrics.plot_confusion_matrix(sgdc_cls, X_test, y_test)
disp.figure_.suptitle("Confusion Matrix")
print("Confusion matrix: \n", disp.confusion_matrix)
```

2. Classification: K-Nearest Neighbors KNN

1) Let's see this algorithm in action with the help of a simple example. Suppose you have a dataset with two variables, which when plotted, looks like the one in the following figure (fig1).

2) Your task is to classify a new data point with **'X'** into **"Blue"** class or **"Red"** class. The coordinate values of the data point are **x=45 and y=50**. Suppose the value of **K is 3**. The KNN algorithm starts by calculating ***the distance of point X from all the points.*** It then finds **the 3 nearest points** with least distance to point **X**. This is shown in the figure below. The three nearest points have been **encircled**.

3) The final step of the KNN algorithm is to assign new point to the class to which **majority of the three nearest points belong**. From the fig2 we can see that the two of the three nearest points belong to the class **"Red"** while one belongs to the class **"Blue"**. Therefore the new data point will be classified as **"Red"**.

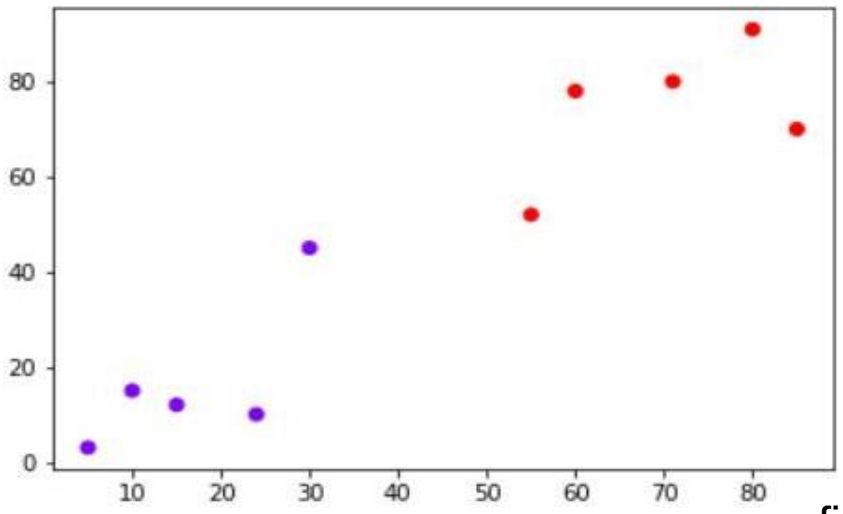


fig1

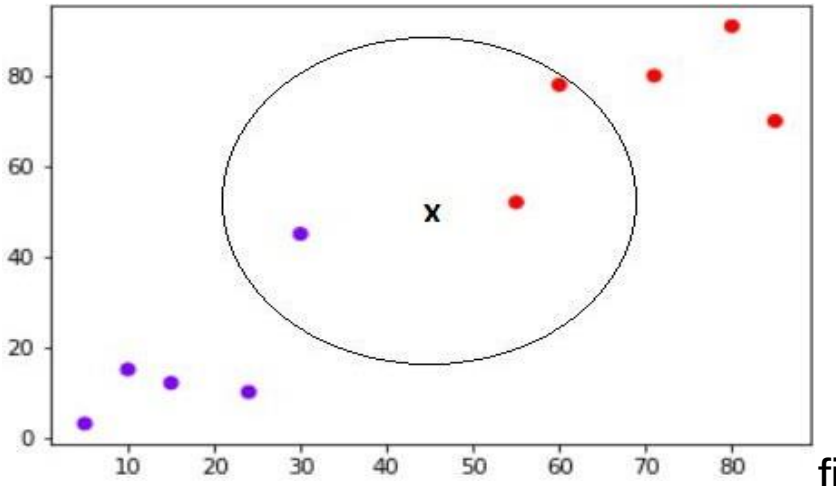


fig2

2. Classification: K-Nearest Neighbors KNN

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.metrics import classification_report,
confusion_matrix
import numpy as np
import matplotlib.pyplot as plt

# Loading data
irisData = load_iris()
# Create feature and target arrays
X = irisData.data
y = irisData.target
# Split into training and test set
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size = 0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors = 7)
knn.fit(X_train, y_train)
predicted = knn.predict(X_test)
print(confusion_matrix(y_test, predicted))
print(classification_report(y_test, predicted))
```

```
neighbors = np.arange(1, 25)
train_accuracy = np.empty(len(neighbors))
test_accuracy  = np.empty(len(neighbors))
# Loop over K values
for i, k in enumerate(neighbors):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)

    # Compute traning and test data accuracy
    train_accuracy[i] = knn.score(X_train, y_train)
    test_accuracy[i] = knn.score(X_test, y_test)

# Generate plot
plt.plot(neighbors, test_accuracy, label = 'Testing dataset
Accuracy')
plt.plot(neighbors, train_accuracy, label = 'Training
dataset Accuracy')
plt.legend()
plt.xlabel('n_neighbors')
plt.ylabel('Accuracy')
plt.show()
```

2. Classification: K-Nearest Neighbors KNN

Advantages:

1) It is extremely **easy** to implement
2) Requires **no training prior** to making **real time** predictions. This makes the KNN algorithm **much faster** than other algorithms that require training e.g SVM, linear regression, etc.
3) New data can be added **seamlessly**.
4) There are only two parameters required to implement KNN i.e. **the value of K** and **the distance function** (e.g. Euclidean or Manhattan etc.)

Disadvantages:

1) The KNN algorithm doesn't work well with **high dimensional data** because with large number of dimensions, it becomes difficult for the algorithm to calculate distance in each dimension.
2) The KNN algorithm **has a high prediction cost** for large datasets. This is because in large datasets the cost of calculating distance between new point and each existing point becomes higher.
3) Finally, the KNN algorithm **doesn't work well with categorical features** since it is difficult to find the distance between dimensions with **categorical features**.

2. Classification: Naive Bayes Classification

Naive Bayes: is a probabilistic machine learning algorithm that can be used in a wide variety of classification tasks. Typical applications include filtering spam, classifying documents, sentiment prediction, etc. It is based on the **Bayes theory**.

Bayes Rule

$$P(Y \mid X) = \frac{P(X \mid Y) * P(Y)}{P(X)}$$

2. Classification: Naive Bayes Classification

The Bayes Rule provides the formula for *the probability of Y given X*. But, in real-world problems, you typically have multiple X variables.

When the features are independent, we can extend the Bayes Rule to what is called Naive Bayes.

When there are multiple X variables, we simplify it by *assuming the X's are independent*, so the **Bayes** rule

$$P(Y=k \mid X) = \frac{P(X \mid Y=k) * P(Y=k)}{P(X)}$$

where, k is a class of Y

becomes, Naive **Bayes**

$$P(Y=k \mid X1..Xn) = \frac{P(X1 \mid Y=k) * P(X2 \mid Y=k) \ldots * P(Xn \mid Y=k) * P(Y=k)}{P(X1) * P(X2) \ldots * P(Xn)}$$

2. Classification: Naive Bayes Classification

Since P(x1, x2, ... , xn) is constant given the input, we can use the following classification rule:

$$P(y|x_1, ..., x_n) \propto P(y) \prod_{i=1}^{n} P(x_i|y)$$

$$y = argmax_y P(y) \prod_{i=1}^{n} P(x_i|y)$$

Laplace Correction :

when you have a model with many features if the entire probability will become zero because one of the feature's value was zero. To avoid this, we increase the count of the variable with zero to a small value (usually 1) in the numerator, so that the overall probability doesn't become zero.

2. Classification: Naive Bayes Classification

	weather	Temperature	Play
1	Sunny	Hot	No
2	Sunny	Hot	No
3	Overcast	Hot	Yes
4	Rainy	Mild	Yes
5	Rainy	Cool	Yes
6	Rainy	Cool	No
7	Overcast	Cool	Yes
8	Sunny	Mild	No
9	Sunny	Cool	Yes
10	Rainy	Mild	Yes
11	Sunny	Mild	Yes
12	Overcast	Mild	Yes
13	Overcast	Hot	No
14	Rainy	Mild	No

weather	No	Yes	P(No)	P(Yes)
Sunny	3	2	3/6	2/8
Overcast	1	3	1/6	3/8
Rainy	2	3	2/6	3/8
Total	6	8	6/14	8/14

Temperature	No	Yes	P(No)	P(Yes)
Hot	3	1	3/6	1/8
Mild	2	4	2/6	4/8
cool	1	3	1/6	3/8
Total	6	8	6/14	8/14

today = (Overcast, Mild)

$$P(Yes/\ today) = P(Overcast/Yes) \times P(Mild/Yes) \times P(Yes)$$

$$P(Yes/\ today) = \frac{3}{8} \times \frac{4}{8} \times \frac{8}{14} = 0,107$$

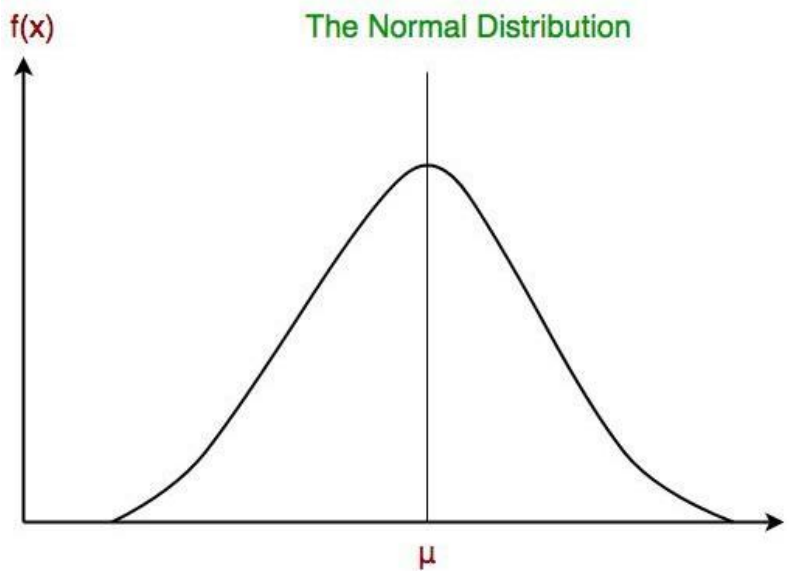
$$P(No/\ today) = P(Overcast/No) \times P(Mild/No) \times P(No)$$

$$P(No/\ today) = \frac{1}{6} \times \frac{2}{6} \times \frac{6}{14} = 0,023$$

$$P(Yes/today) = \frac{0,107}{0,107 + 0,023} = 0,82 \qquad P(No/today) = \frac{0,023}{0,107 + 0,023} = 0,18$$

2. Classification: Gaussian Naive Bayes Classification

In Gaussian Naive Bayes, **continuous values** associated with each feature are **assumed** to be distributed according to a **Gaussian distribution**. A Gaussian distribution is also called Normal distribution.



The likelihood of the features is assumed to be Gaussian, hence, conditional probability is given by:

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} exp\left(-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}\right)$$

2. Classification: Gaussian Naive Bayes Classification

```
# Assigning features and label variables
weather=['Sunny','Sunny','Overcast','Rainy','Rainy','R
ainy','Overcast','Sunny','Sunny',
'Rainy','Sunny','Overcast','Overcast','Rainy']
temp=['Hot','Hot','Hot','Mild','Cool','Cool','Cool','Mild','
Cool','Mild','Mild','Mild','Hot','Mild']
play=['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Ye
s','Yes','Yes','No','No']

# Import LabelEncoder
from sklearn import preprocessing
# creating labelEncoder
le = preprocessing.LabelEncoder()
# Converting string labels into numbers.
weather_encoded=le.fit_transform(weather)
print("weather:",weather_encoded)
# Converting string labels into numbers
temp_encoded=le.fit_transform(temp)
label=le.fit_transform(play)
print("Temp:    ",temp_encoded)
print("Play:    ",label)
```

```
# Combinig weather and temp into single listof tuples
features = zip(weather_encoded,temp_encoded)
import numpy as np
features = np.asarray(list(features))
print("features : ",features)

#Import Gaussian Naive Bayes model
from sklearn.naive_bayes import GaussianNB
#Create a Gaussian Classifier
model = GaussianNB()
# Train the model using the training sets
model.fit(features,label)
# Predict Output
predicted= model.predict([[0,2]]) # 0:Overcast, 2:Mild
print ("Predicted Value (No = 0, Yes = 1):", predicted)
```

2. Classification: Gaussian Naive Bayes Classification

```
# load the iris dataset
from sklearn.datasets import load_iris
iris = load_iris()
# store the feature matrix (X) and response vector (y)
X = iris.data
y = iris.target
# splitting X and y into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=100)
# training the model on training set
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
gnb.fit(X_train, y_train)
# making predictions on the testing set
y_pred = gnb.predict(X_test)
# comparing actual response values (y_test) with predicted response values (y_pred)
from sklearn import metrics
print("Gaussian Naive Bayes model accuracy(in %):", metrics.accuracy_score(y_test, y_pred)*100)
print("Number of mislabeled points out of a total %d points : %d" % (X_test.shape[0], (y_test !=
y_pred).sum()))
```

2. Classification: Multinomial Naive Bayes Classification

The multinomial Naive Bayes classifier is suitable for classification with **discrete features** (e.g., word counts for text classification). The multinomial distribution normally requires **integer feature counts**. However, in practice, fractional counts such as tf-idf may also work.

Advantages:

1) They are extremely **fast** for both training and prediction.
2) They provide a **straightforward probabilistic** prediction.
3) They are often **very easily** interpretable.
4) They have very **few (if any) tunable parameters**.

2. Classification: Multinomial Naive Bayes Classification

```
from sklearn.datasets import fetch_20newsgroups
data = fetch_20newsgroups()
print(data.target_names)
# For simplicity here, we will select just a few of these categories
categories = ['talk.religion.misc', 'soc.religion.christian',
              'sci.space', 'comp.graphics']
sub_data = fetch_20newsgroups(subset='train',
categories=categories)
X, Y = sub_data.data, sub_data.target
print("Examples = ",len(X)," Labels = ", len(Y))
# Here is a representative entry from the data:
print(X[5])
# In order to use this data for machine learning, we need to be
able to convert the content of each string into a vector of numbers.
# For this we will use the TF-IDF vectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
vec = TfidfVectorizer()
vec.fit(X)
texts = vec.transform(X).toarray()
print(" texts shape = ",texts.shape)
```

```
# split the dataset into training data and test data
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(texts, Y,
                    test_size= 0.20, random_state=100, stratify=Y)
# training the model on training set
from sklearn.naive_bayes import MultinomialNB
clf = MultinomialNB()
clf.fit(X_train, Y_train)
# making predictions on the testing set
predicted = clf.predict(X_test)

from sklearn import metrics
print("Classification report : \n", clf,"\n",
metrics.classification_report(Y_test, predicted))
disp = metrics.plot_confusion_matrix(clf, X_test, Y_test)
disp.figure_.suptitle("Confusion Matrix")
print("Confusion matrix: \n", disp.confusion_matrix)
```

2. Classification: Decision Trees

Decision trees are supervised learning algorithms used for both, classification and regression tasks.

The main idea of decision trees is to **find the best descriptive features** which contain the most information regarding the target feature and **then split the dataset along the values of these features** such that the target feature values for the resulting **sub_datasets** are as pure as possible.

The descriptive feature which leaves the target feature most purely is said to be the most informative one. **This process of finding the most informative feature is done until we accomplish a stopping criterion** where we then finally end up in so-called **leaf nodes**. The leaf nodes contain the predictions we will make for new query instances presented to our trained model. This is possible since the model has kind of learned the underlying structure of the training data and hence can, given some assumptions, make predictions about the target feature value (class) of unseen query instances.

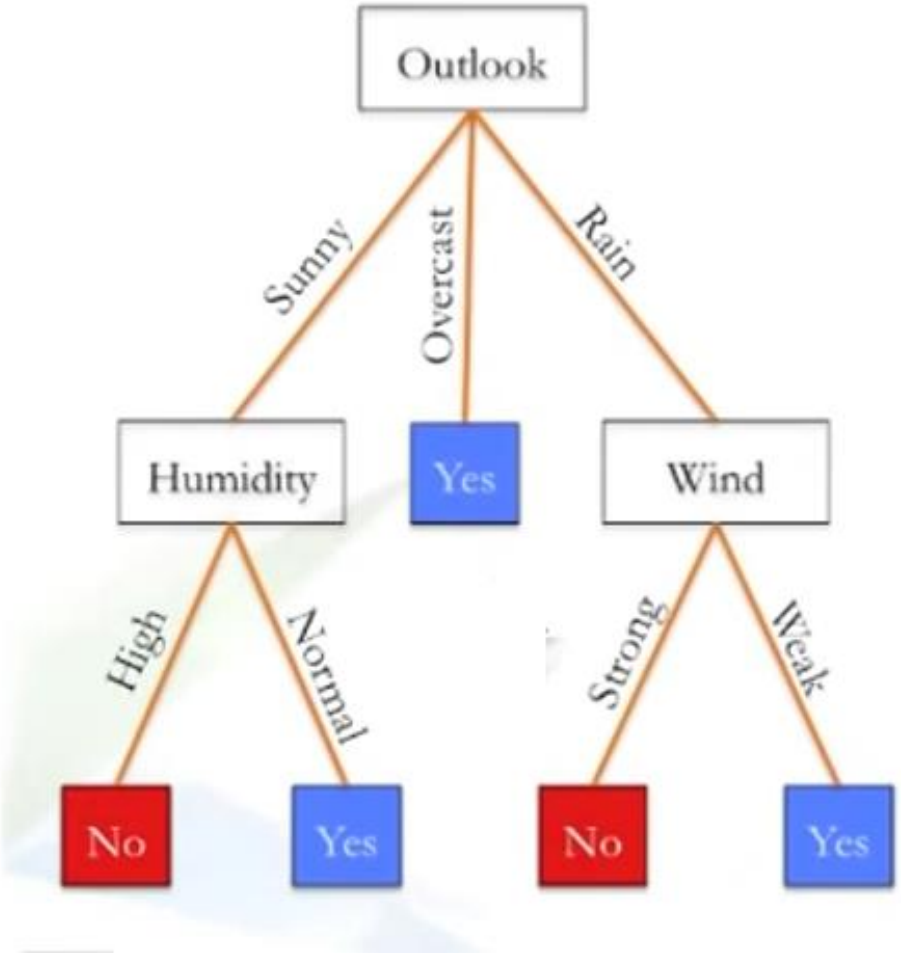
Decision trees can handle both categorical and numerical data.

2. Classification: Decision Trees

Rafael Nadal

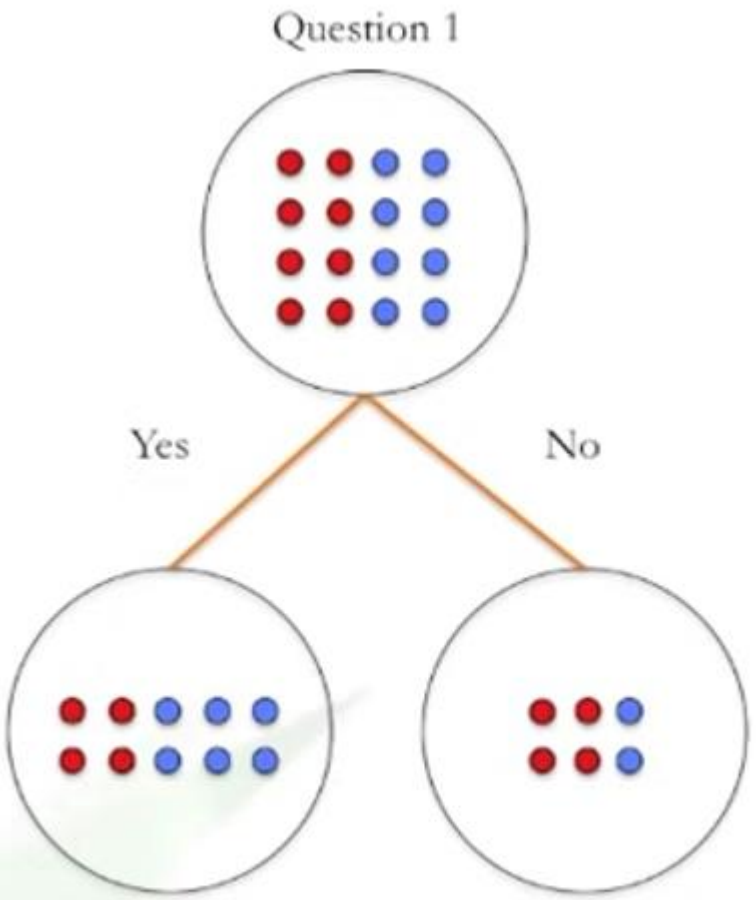
Will Nadal play tennis?

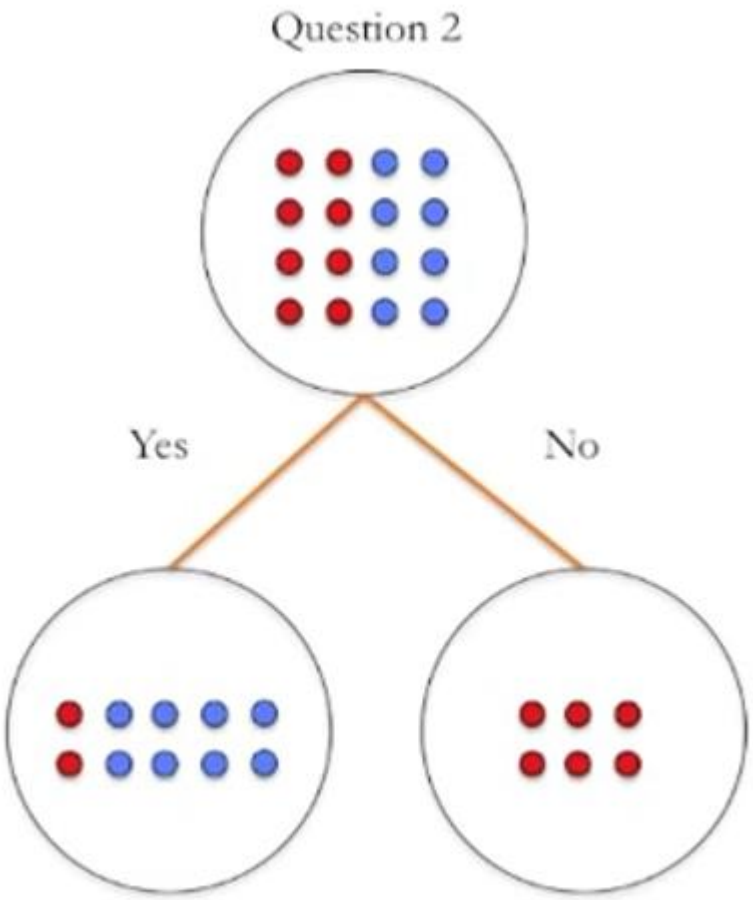
2. Classification: Decision Trees



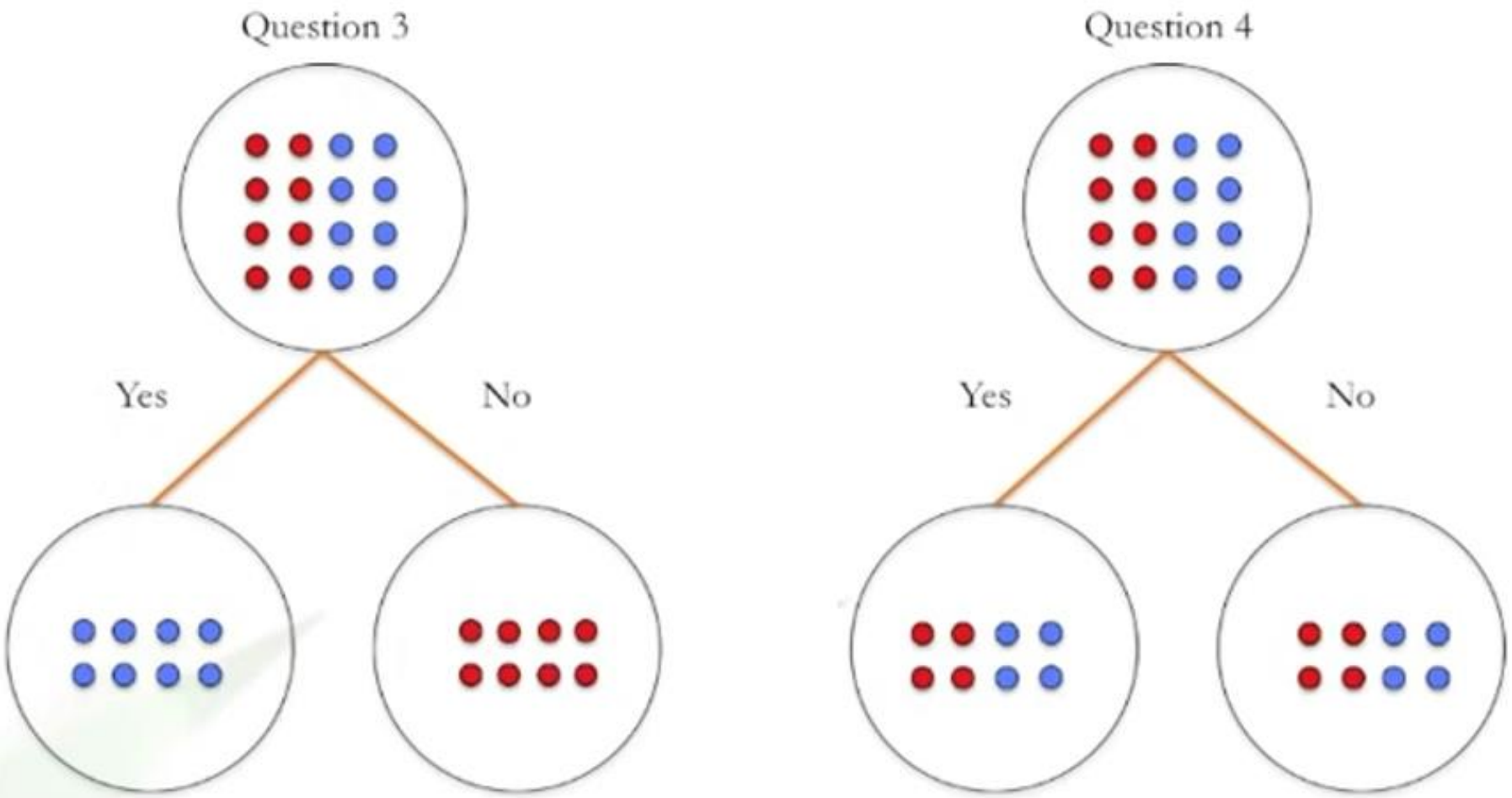
Day	Outlook	Temp	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Weak	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Strong	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

2. Classification: Decision Trees





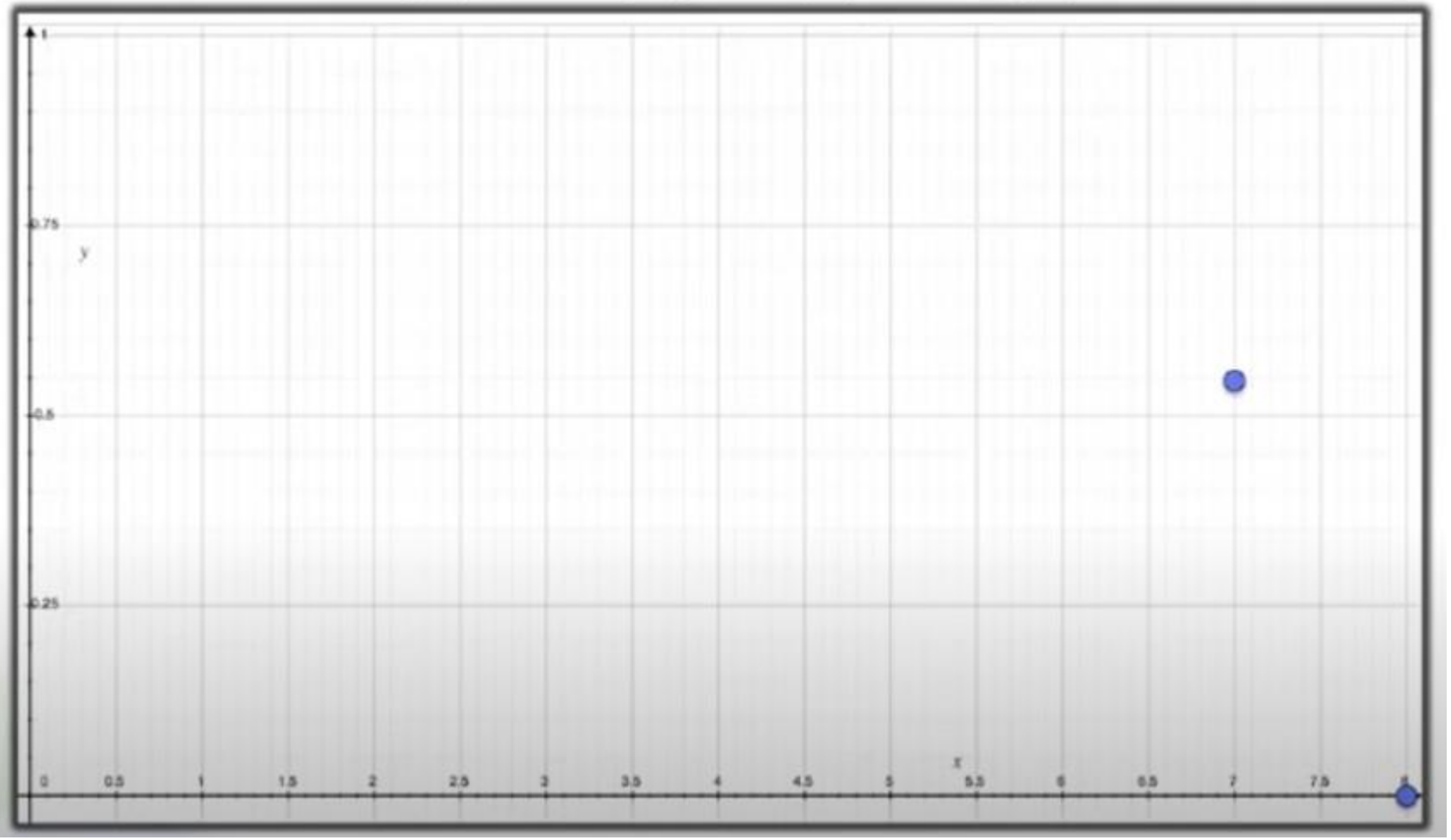
2. Classification: Decision Trees



$$E(S) = -p_+ log(p_+) - p_- log(p_-)$$

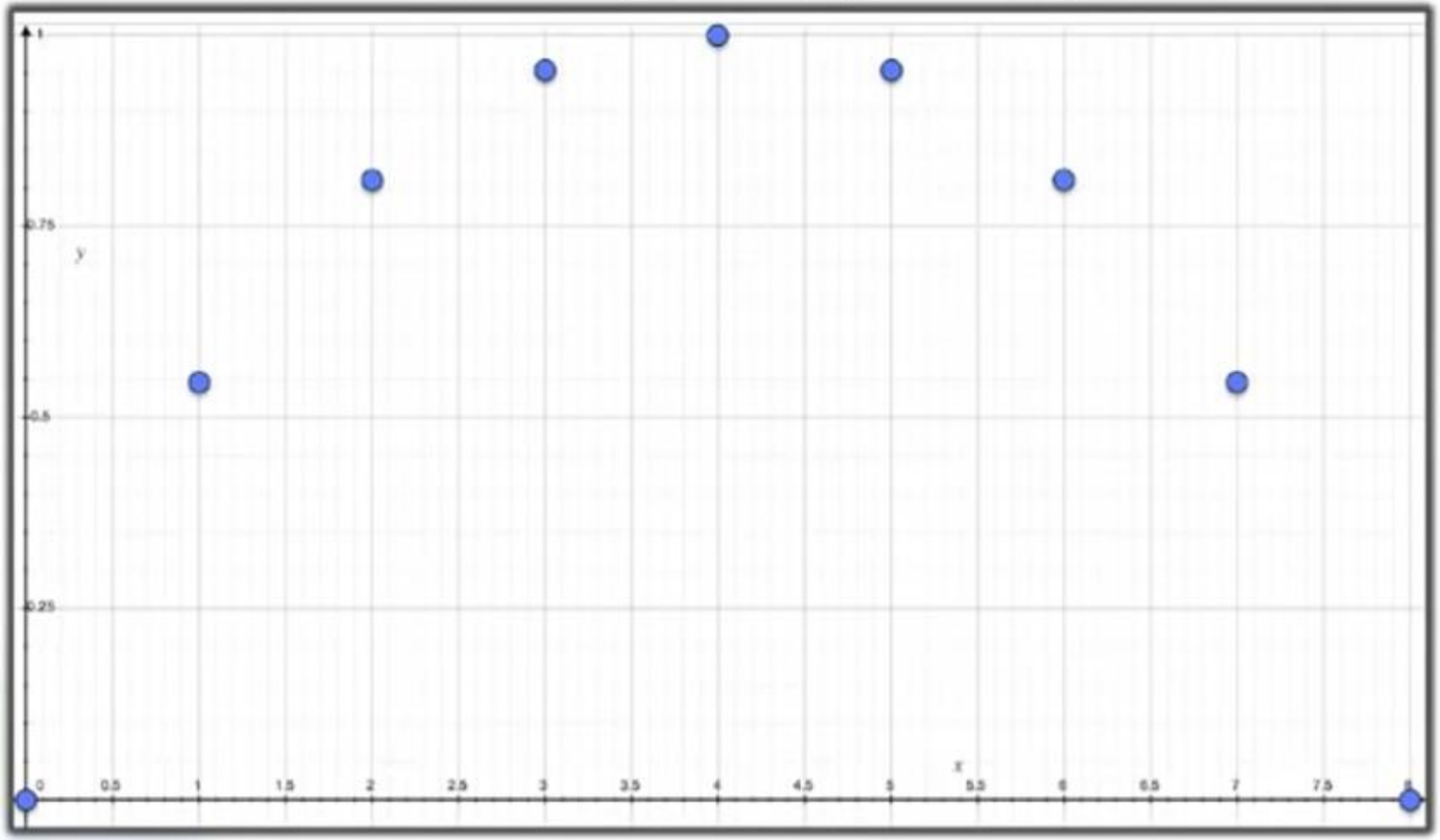
2. Classification: Decision Trees

$$-\left(\frac{7}{8}\right) log_2\left(\frac{7}{8}\right) - \left(\frac{1}{8}\right) log_2\left(\frac{1}{8}\right) = 0.54$$



2. Classification: Decision Trees

$$-\left(\frac{0}{8}\right) log_2\left(\frac{0}{8}\right) - \left(\frac{8}{8}\right) log_2\left(\frac{8}{8}\right) = 0$$

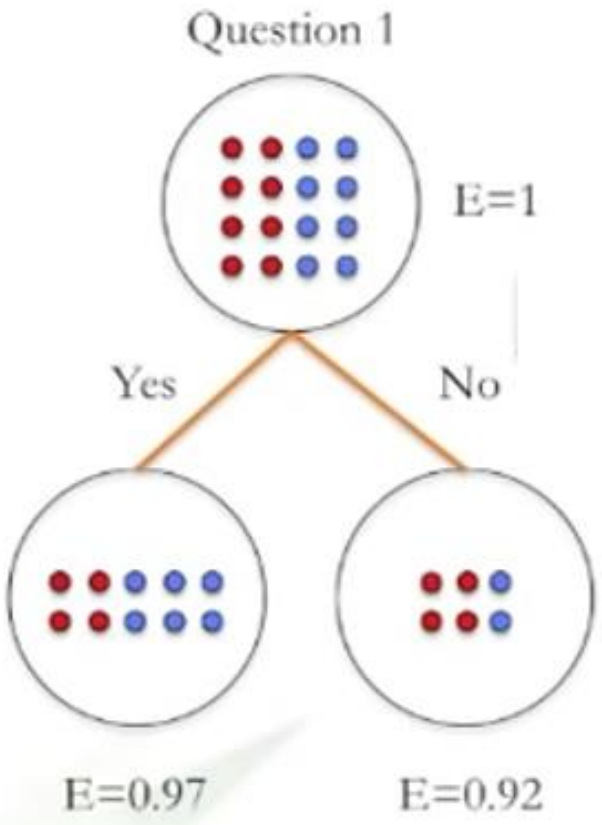


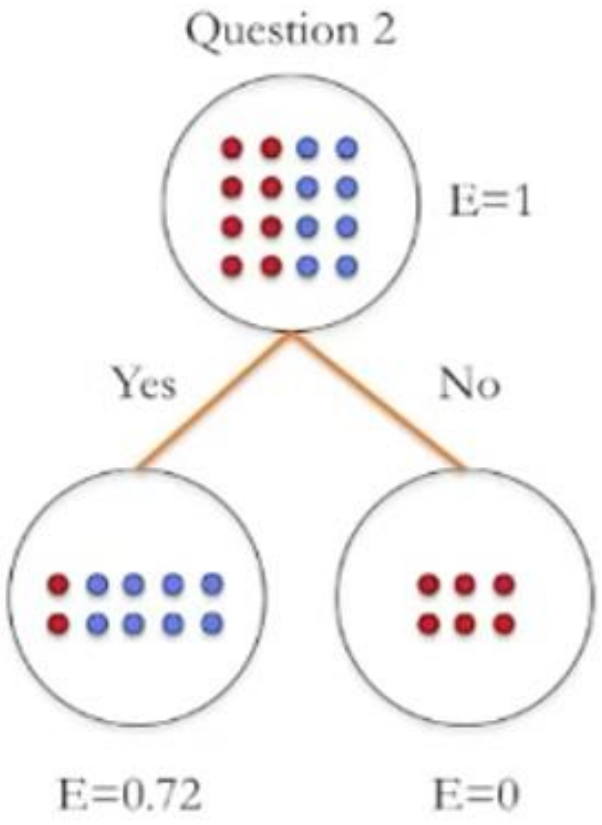
2. Classification: Decision Trees

$$y = -\sum_{i=1}^{k} p_i log_k(p_i)$$

$$y = -\underbrace{\left[\left(\frac{1}{10}\right) log_4\left(\frac{1}{10}\right)\right]}_{\text{Red}} - \underbrace{\left[\left(\frac{3}{10}\right) log_4\left(\frac{3}{10}\right)\right]}_{\text{Green}} - \underbrace{\left[\left(\frac{2}{10}\right) log_4\left(\frac{2}{10}\right)\right]}_{\text{Blue}} - \underbrace{\left[\left(\frac{4}{10}\right) log_4\left(\frac{4}{10}\right)\right]}_{\text{Yellow}}$$

2. Classification: Decision Trees

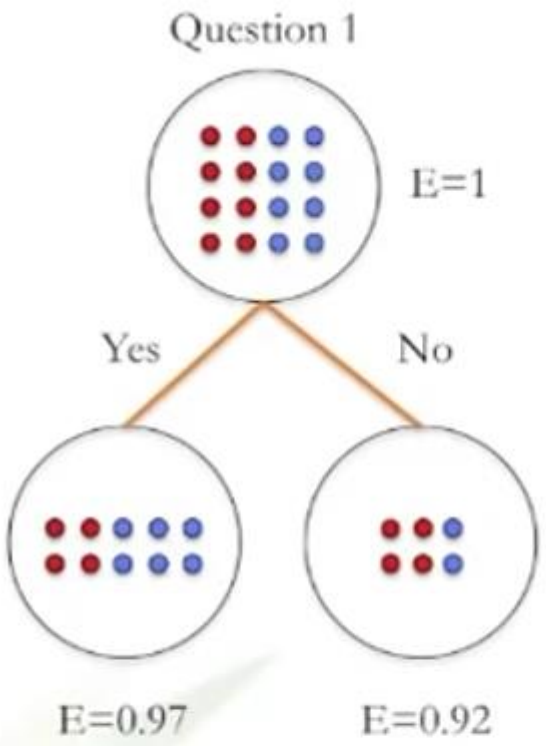


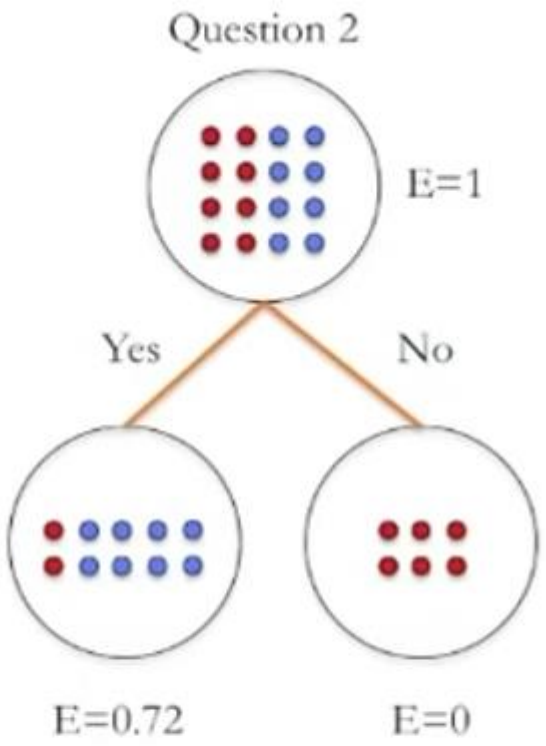


Information Gain

$$G(S,Q) = E(S) - \sum_{i=1}^{k} p_i E(S,Q_i)$$

2. Classification: Decision Trees





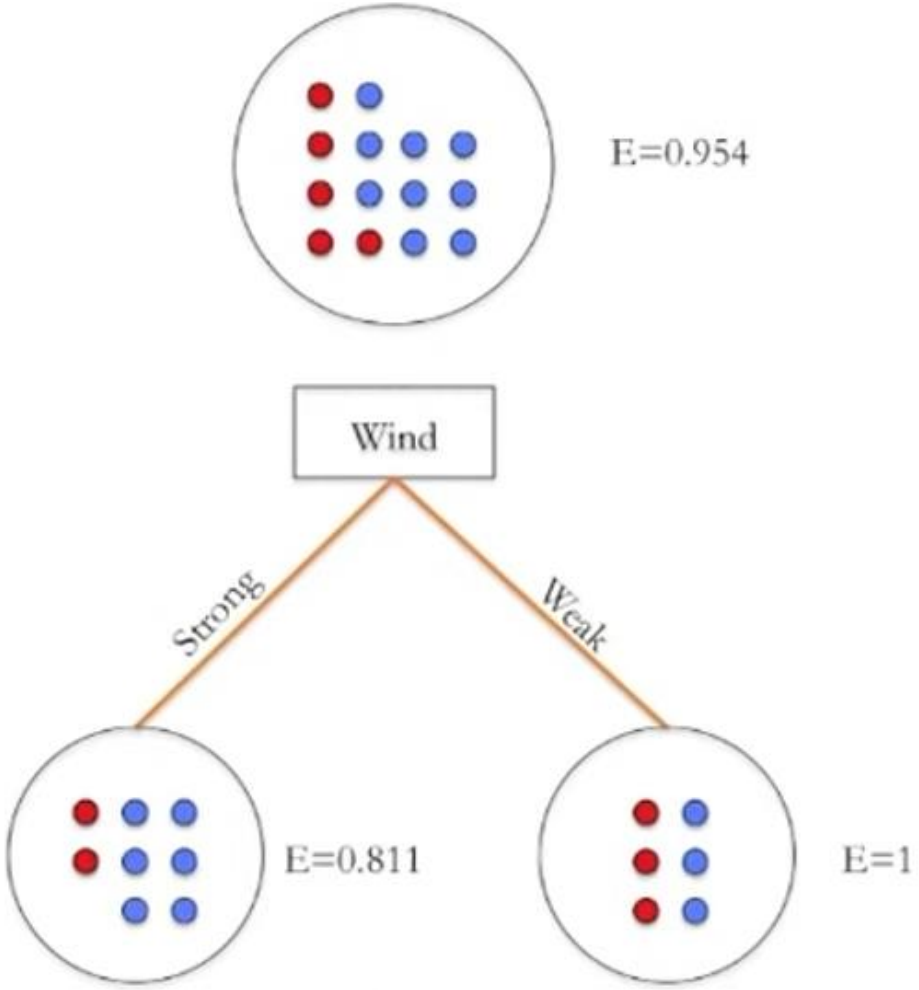
$$G(S, Q_1) = 1 - \left(\frac{10}{16}\right) \times 0.97 - \left(\frac{6}{16}\right) \times 0.92$$

$$G(S, Q_1) = 0.049$$

$$G(S, Q_2) = 1 - \left(\frac{10}{16}\right) \times 0.72 - \left(\frac{6}{16}\right) \times 0$$

$$G(S, Q_2) = 0.55$$

2. Classification: Decision Trees

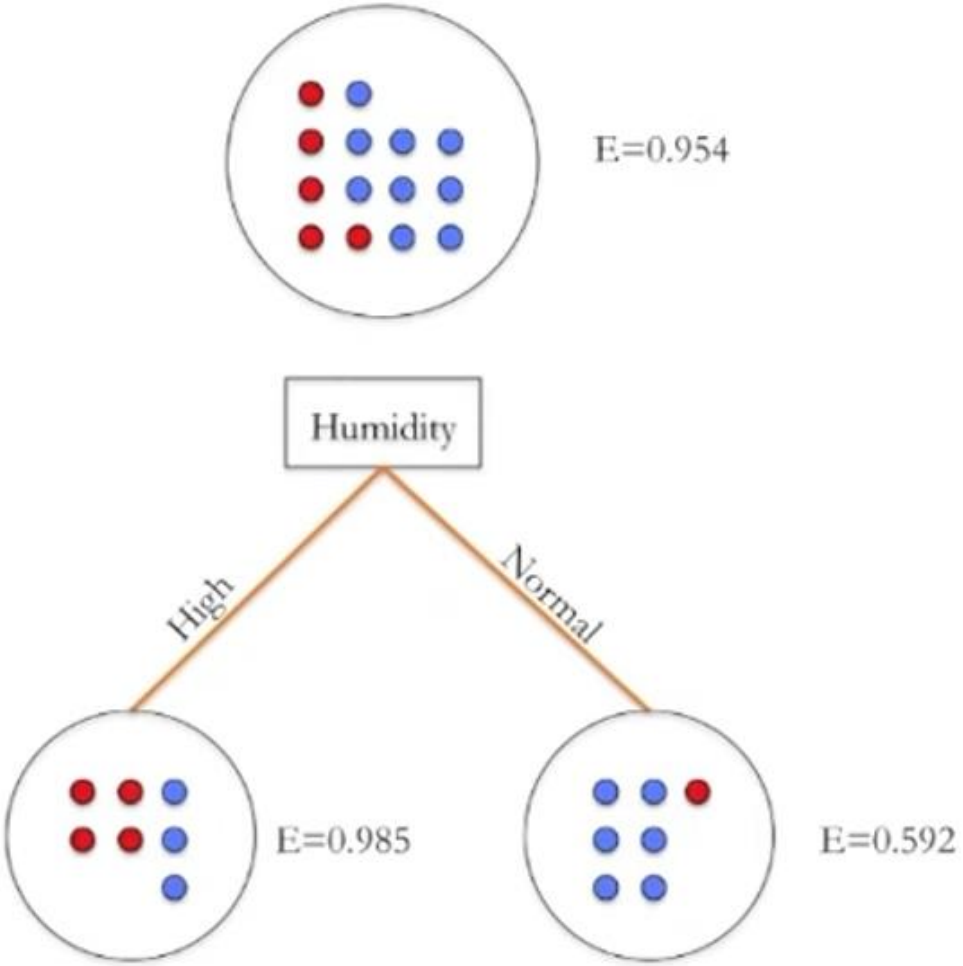


$$G(S, Wind) = 0.954 - \frac{8}{14}0.811 - \frac{6}{14}1$$

$$G(S, Wind) = 0.048$$

2. Classification: Decision Trees

$G(S, Wind) = 0.048$



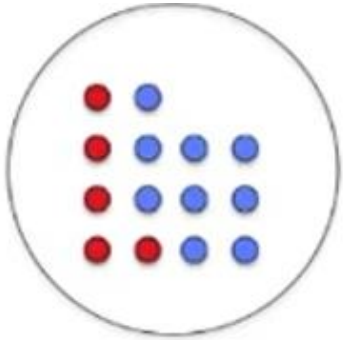
$$G(S, Humidity) = 0.954 - \frac{7}{14}0.985 - \frac{7}{14}0.592$$

$$G(S, Humidity) = 0.151$$

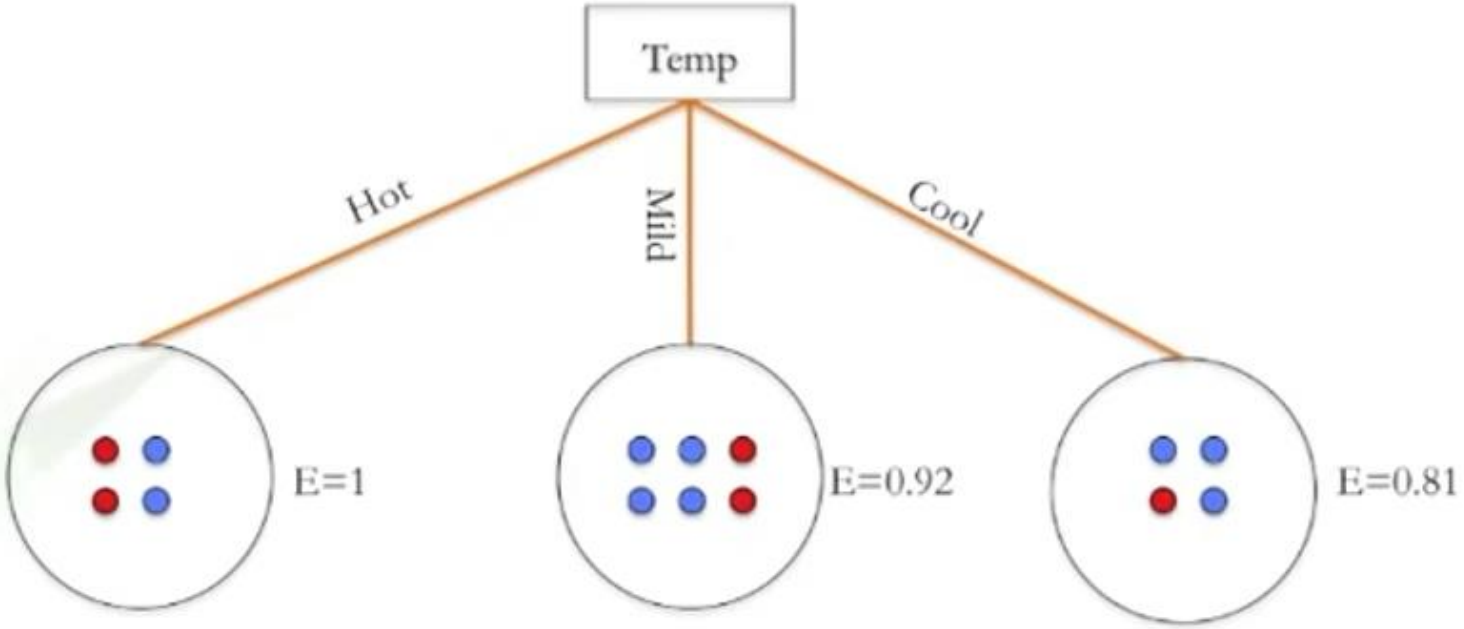
2. Classification: Decision Trees

$G(S, Wind) = 0.048$

$G(S, Humidity) = 0.151$



E=0.954



$$G(S, Temp) = 0.954 - \frac{4}{14}1 - \frac{6}{14}0.92 - \frac{4}{14}0.81$$

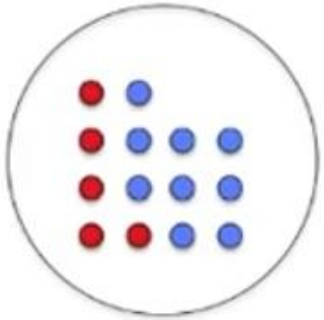
$$G(S, Temp) = 0.042$$

2. Classification: Decision Trees

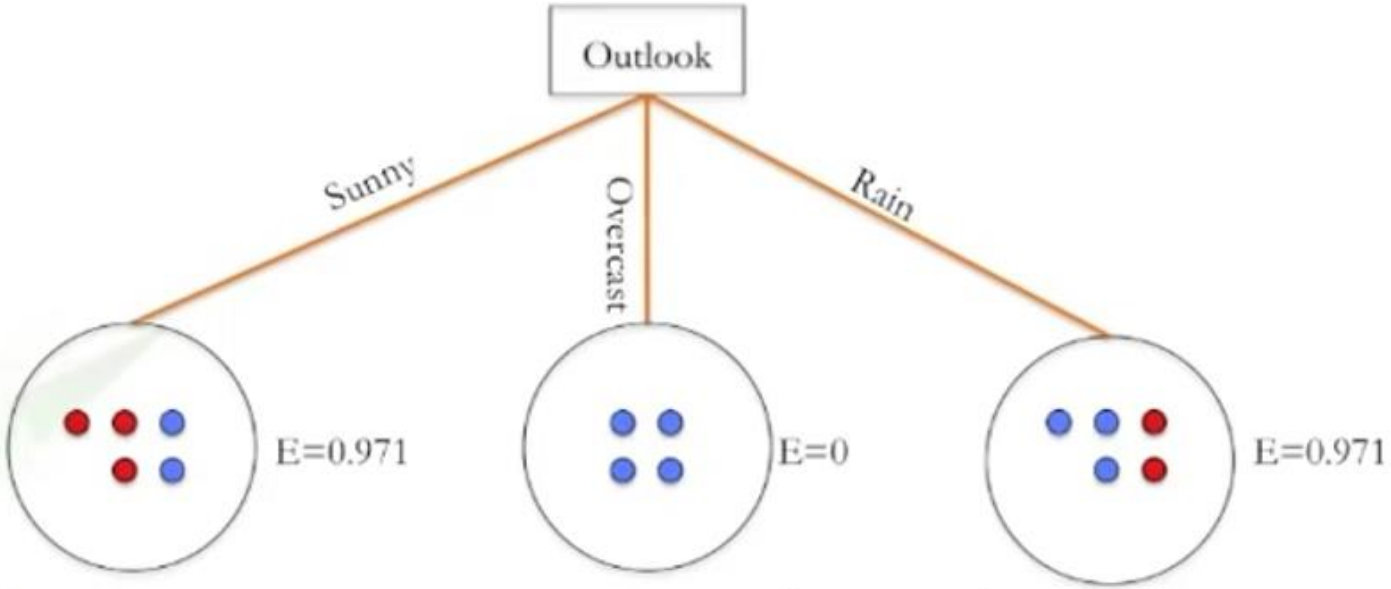
$G(S, Wind) = 0.048$

$G(S, Humidity) = 0.151$

$G(S, Temp) = 0.042$



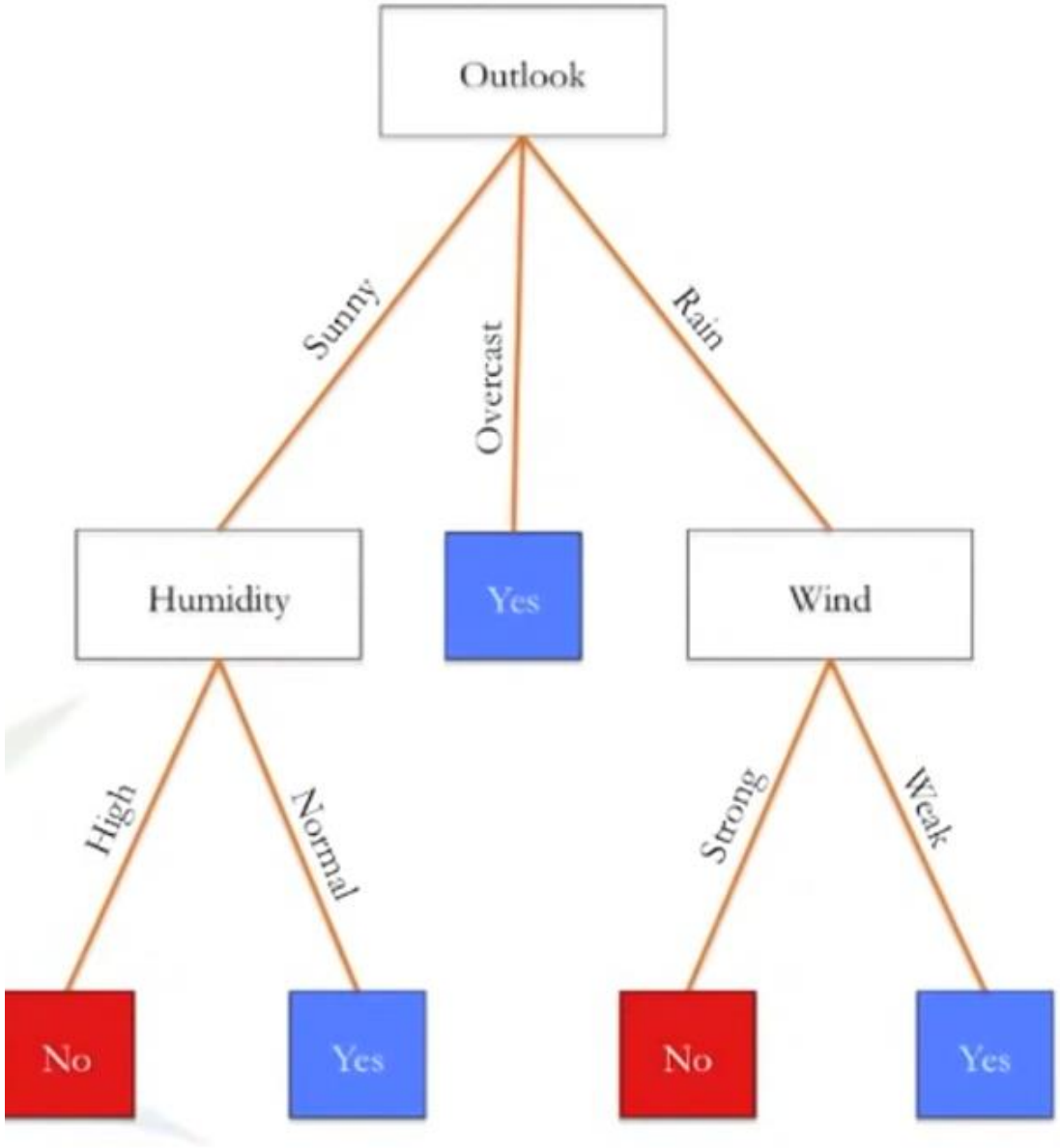
E=0.954



$$G(S, Outlook) = 0.954 - \frac{5}{14}0.971 - \frac{4}{14}0 - \frac{5}{14}0.971$$

$$G(S, Outlook) = 0.247$$

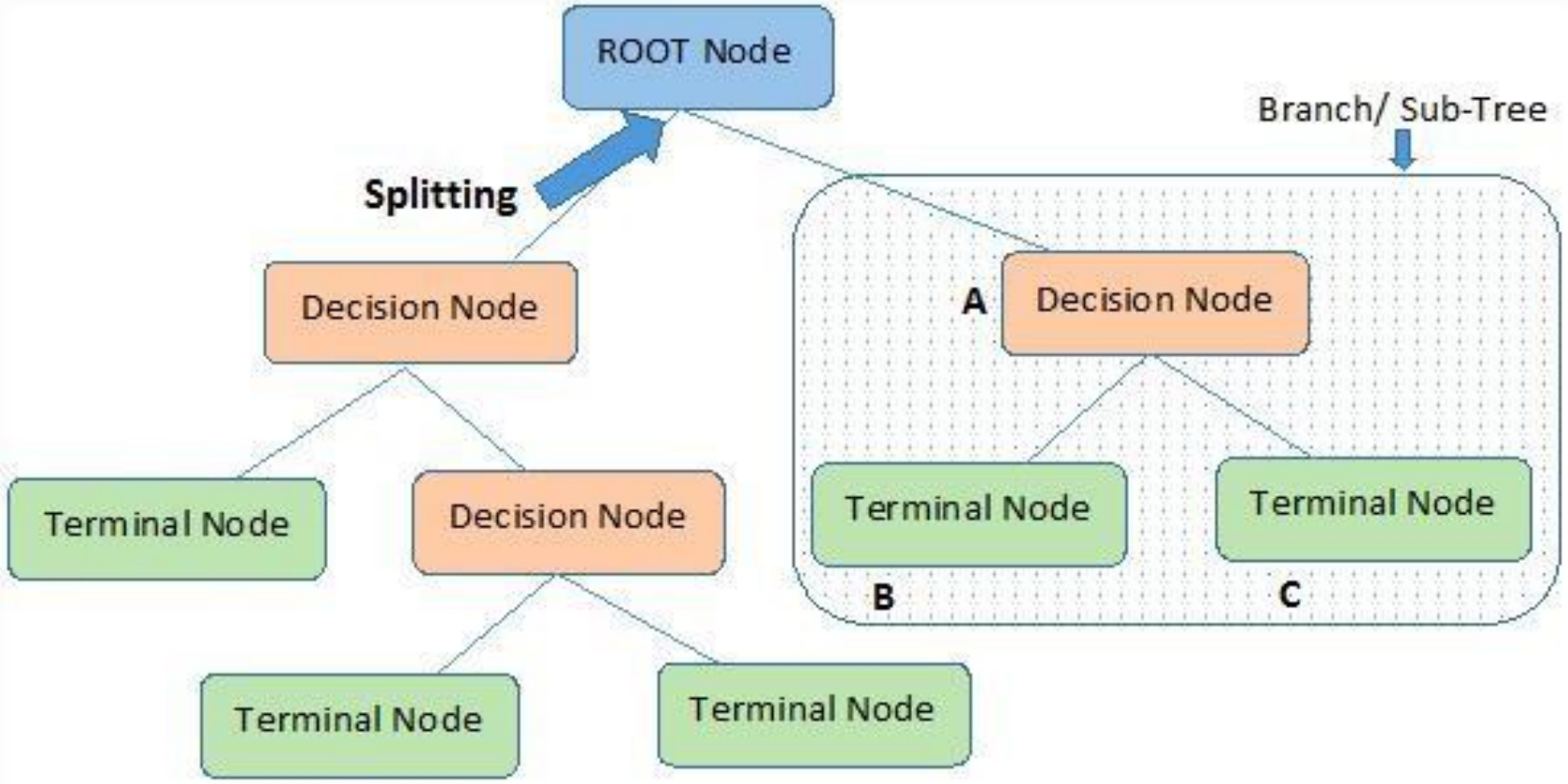
2. Classification: Decision Trees



2. Classification: Decision Trees

1) **Root Node:** This attribute is used for dividing the data into two or more sets. The feature attribute in this node is selected based on Attribute Selection Techniques.
2) **Branch or Sub-Tree:** A part of the entire decision tree is called branch or sub-tree.
3) **Splitting:** Dividing a node into two or more sub-nodes based on **if-else** conditions.
4) **Decision Node:** After splitting the sub-nodes into further sub-nodes, then it is called as the decision node.
5) **Leaf or Terminal Node:** This is the end of the decision tree where it cannot be split into further sub-nodes.

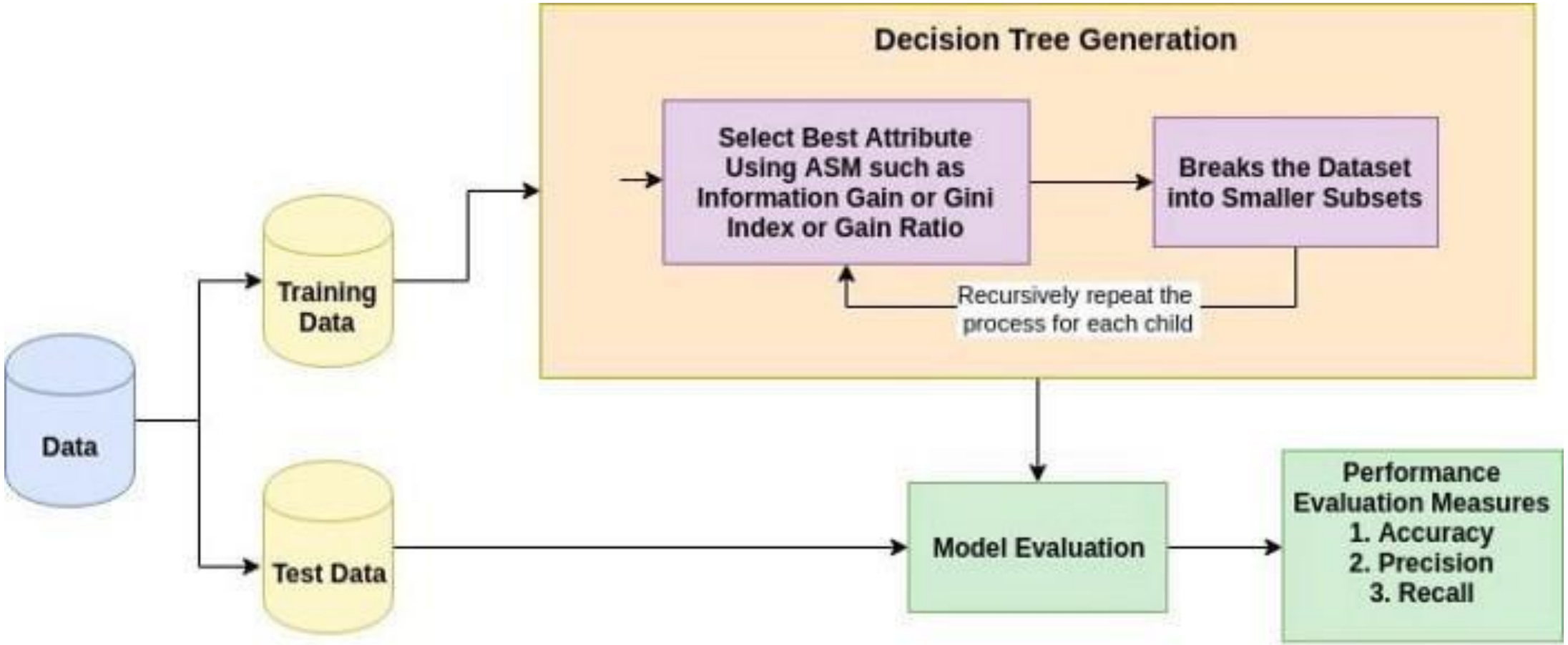
2. Classification: Decision Trees



Note:- A is parent node of B and C.

2. Classification: Decision Trees

How does the Decision Tree algorithm work?



2. Classification: Decision Trees

- **The root node feature** is selected based on the results from the Attribute Selection Measure(ASM).

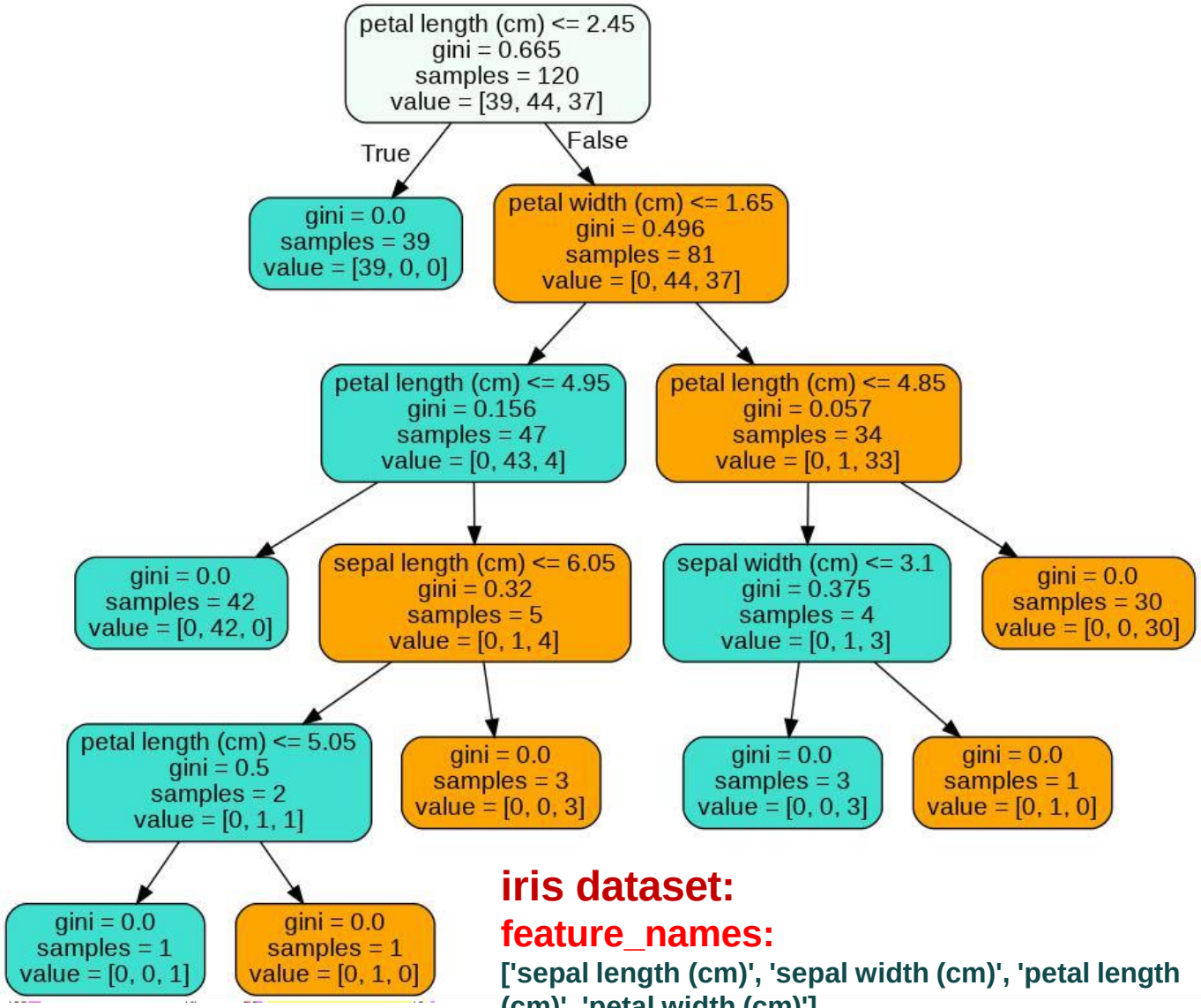
- **Attribute selection measure** (ASM) is a **heuristic** for selecting the splitting criterion that partition the data into the best possible manner.

- The ASM is repeated until there is a leaf node or a terminal node where it cannot be split into sub-nodes.

- The two main ASM techniques are: **Gini impurity** and **Entropy**

2. Classification: Decision Trees - Example

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.65
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- sepal length (cm) <= 6.05
|   |   |   |   |--- petal length (cm) <= 5.05
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- petal length (cm) >  5.05
|   |   |   |   |   |--- class: 1
|   |   |   |--- sepal length (cm) >  6.05
|   |   |   |   |--- class: 2
|   |--- petal width (cm) >  1.65
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal width (cm) <= 3.10
|   |   |   |   |--- class: 2
|   |   |   |--- sepal width (cm) >  3.10
|   |   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: 2
```



iris dataset:

feature_names:

['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

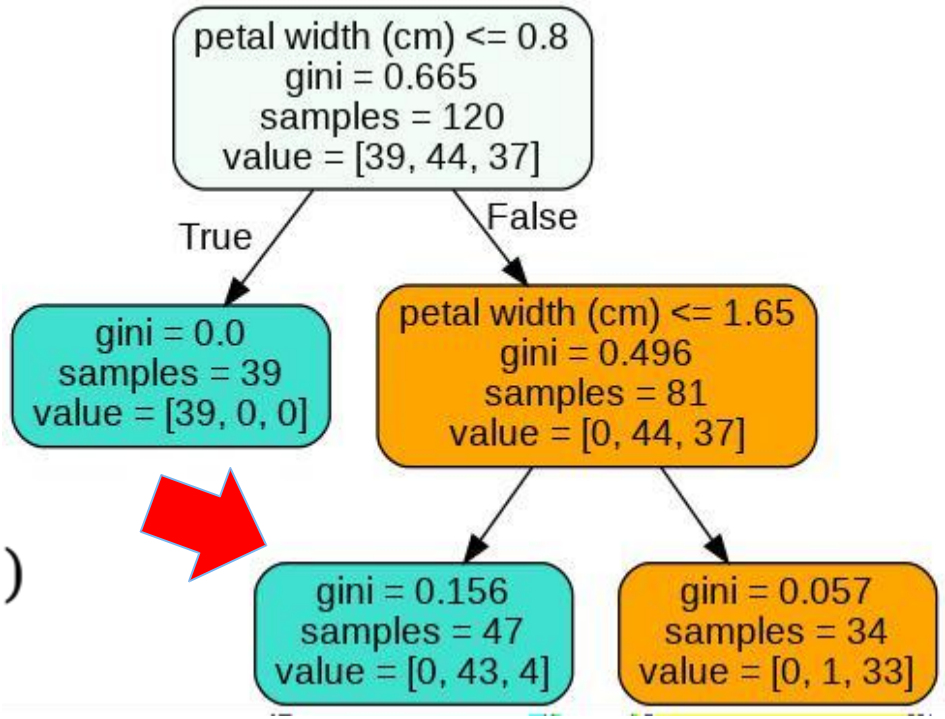
target_names: ['setosa' 'versicolor' 'virginica']

2. Classification: Decision Trees

Gini impurity:

$$G_i = 1 - \sum_{k=1}^{n} \frac{P_i}{k}^2$$

$$GI = 1 - \left(\left(\frac{0}{47}^2\right) + \left(\frac{43}{47}\right)^2 + \left(\frac{4}{47}\right)^2\right) = 0,156$$



Entropy:

$$H_i = -\sum_{\substack{k=1 \\ \frac{P_i}{k} \neq 0}}^{n} \frac{P_i}{k} \times log_2\left(\frac{P_i}{k}\right)$$

$$H_i = -\frac{43}{47} \times log_2\left(\frac{43}{47}\right) - \frac{4}{47} \times log_2\left(\frac{4}{47}\right)$$

2. Classification: Decision Trees

Estimating Class Probabilities:

A Decision Tree can also estimate the probability that an instance belongs to a particular class k: first it traverses the tree to find the leaf node for this instance, and then it returns the ratio of training instances of class k in this node.

```
print("predict_proba : ",tree_clf.predict_proba([[1, 1.5, 3.2, 0.5]]))
```

```
predict_proba :  [[0.        0.91489362 0.08510638]]
```

Scikit-Learn uses the **CART algorithm**, which produces only **binary trees**: nonleaf nodes always have two children (i.e., questions only have **yes/no** answers). However, other algorithms such as **ID3** can produce Decision Trees with nodes that **have more than two children**.

2. Classification: Decision Trees

Advantages of CART

- Simple to understand, interpret, visualize.
- Decision trees implicitly perform variable screening or feature selection.
- Can handle both numerical and categorical data. Can also handle multi-output problems.
- Decision trees require relatively little effort from users for data preparation.
- Nonlinear relationships between parameters do not affect tree performance.

Disadvantages of CART

- Decision-tree learners can create over-complex trees that do not generalize the data well. This is called overfitting.
- Decision trees can be unstable because small variations in the data might result in a completely different tree being generated. This is called variance, which needs to be lowered by methods like bagging and boosting.
- Greedy algorithms cannot guarantee to return the globally optimal decision tree. This can be mitigated by training multiple trees, where the features and samples are randomly sampled with replacement.
- Decision tree learners create biased trees if some classes dominate. It is therefore recommended to balance the dataset prior to fitting with the decision tree.

2. Classification: Decision Trees

Optimizing the performance of the trees :

1) **max_depth**: The maximum depth of the tree.
2) **criterion**: This parameter takes the criterion method as the value. The default value is Gini.
3) **splitter**: This parameter allows us to choose the split strategy. Best and random are available types of the split. The default value is best.

How to avoid Over-Fitting?

Use Random Forest trees.

2. Classification: Decision Trees - Example

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
# load the iris dataset
dat = load_iris()
# store the feature matrix (X) and response vector (y)
X = dat.data
y = dat.target
# splitting X and y into training and testing sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)
# training the model on training set
# criterion='gini' splitter='best' max_depth=2
tree_clf = DecisionTreeClassifier(max_depth=2)
tree_clf.fit(X_train, y_train)
# making predictions on the testing set
predicted = tree_clf.predict(X_test)
# comparing actual response values (y_test) with predicted response values (predicted)
from sklearn import metrics
print("Classification report : \n", tree_clf,"\n", metrics.classification_report(y_test, predicted))
disp = metrics.plot_confusion_matrix(tree_clf, X_test, y_test)
disp.figure_.suptitle("Confusion Matrix")
print("Confusion matrix: \n", disp.confusion_matrix)
# plot the tree with the plot_tree function:
from sklearn import tree
tree.plot_tree(tree_clf.fit(X_train, y_train))
```

2. Classification: Decision Trees - Example

```
# Visualize data
import collections
import pydotplus
dot_data = tree.export_graphviz(tree_clf, feature_names=dat.feature_names, out_file=None, filled=True, rounded=True)
graph = pydotplus.graph_from_dot_data(dot_data)
colors = ('turquoise', 'orange')
edges = collections.defaultdict(list)
for edge in graph.get_edge_list():
    edges[edge.get_source()].append(int(edge.get_destination()))

for edge in edges:
    edges[edge].sort()
    for i in range(2):
        dest = graph.get_node(str(edges[edge][i]))[0]
        dest.set_fillcolor(colors[i])

graph.write_png('tree1.png')
#pdf
import graphviz
dot_data = tree.export_graphviz(tree_clf, out_file=None)
graph = graphviz.Source(dot_data)
graph.render("iris")
dot_data = tree.export_graphviz(tree_clf, out_file=None, feature_names=dat.feature_names, class_names=dat.target_names, filled=True, rounded=True,
special_characters=True)
graph = graphviz.Source(dot_data)
graph.view('tree2.pdf')

# The tree can also be exported in textual format
from sklearn.tree import export_text
r = export_text(tree_clf, feature_names=dat.feature_names)
print(r)
```

3. Ensemble Methods: Random Forest Classifier

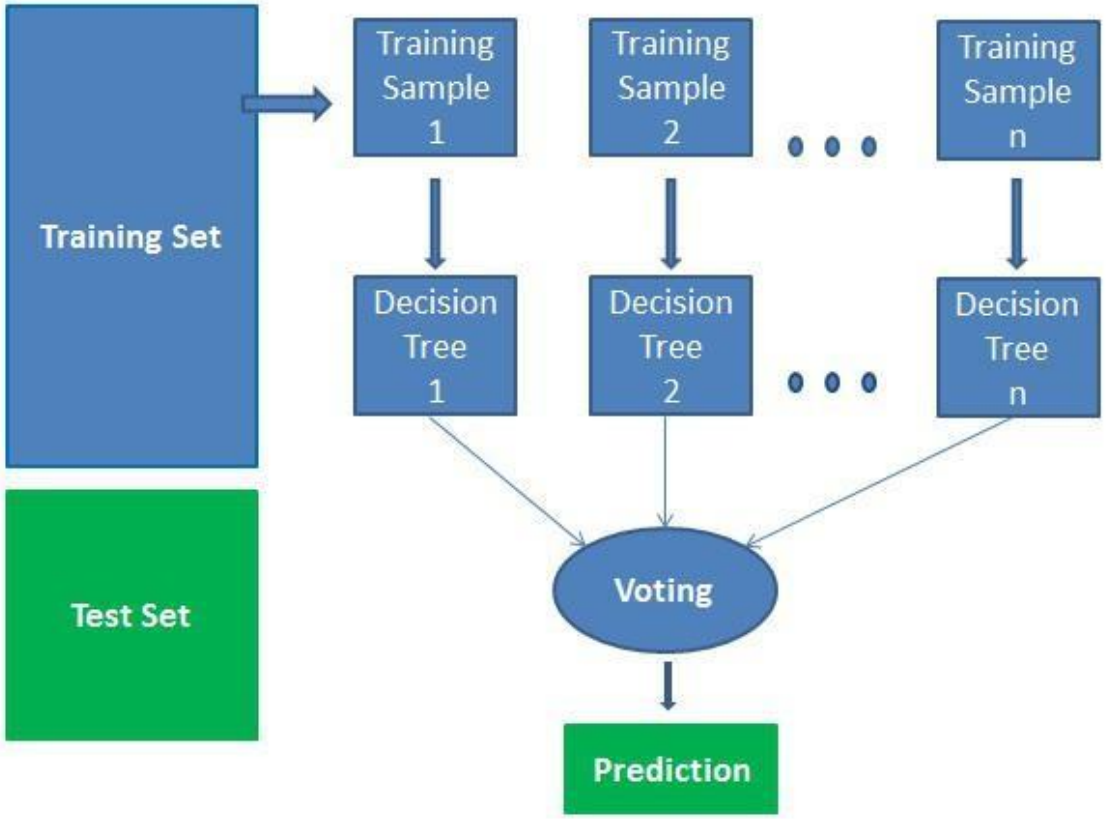
The random forest classifier is a supervised learning algorithm and composed of multiple decision trees. By averaging out the impact of several decision trees, random forests tend to improve prediction.

1) Select random samples from a given dataset.

2) Construct a decision tree for each sample and get a prediction result f rom each decision tree.

3) Perform a vote for each predicted result.

4) Select the prediction result with the most votes as the final prediction,



3. Ensemble Methods: Random Forest Classifier

Advantages:

1) Random forests is considered as a **highly accurate and robust method** because of the number of decision trees participating in the process.
2) It **does not suffer from the overfitting problem**. The main reason is that it takes the average of all the predictions, which cancels out the biases.
3) The algorithm can be used in both **classification** and **regression** problems.

Disadvantages:

1) Random forests is **slow** in generating predictions because it has multiple decision trees. Whenever it makes a prediction, all the trees in the forest have to make a prediction for the same given input and then perform voting on it. This whole process is time-consuming.
2) The model is **difficult to interpret** compared to a decision tree, where you can easily make a decision by following the path in the tree.

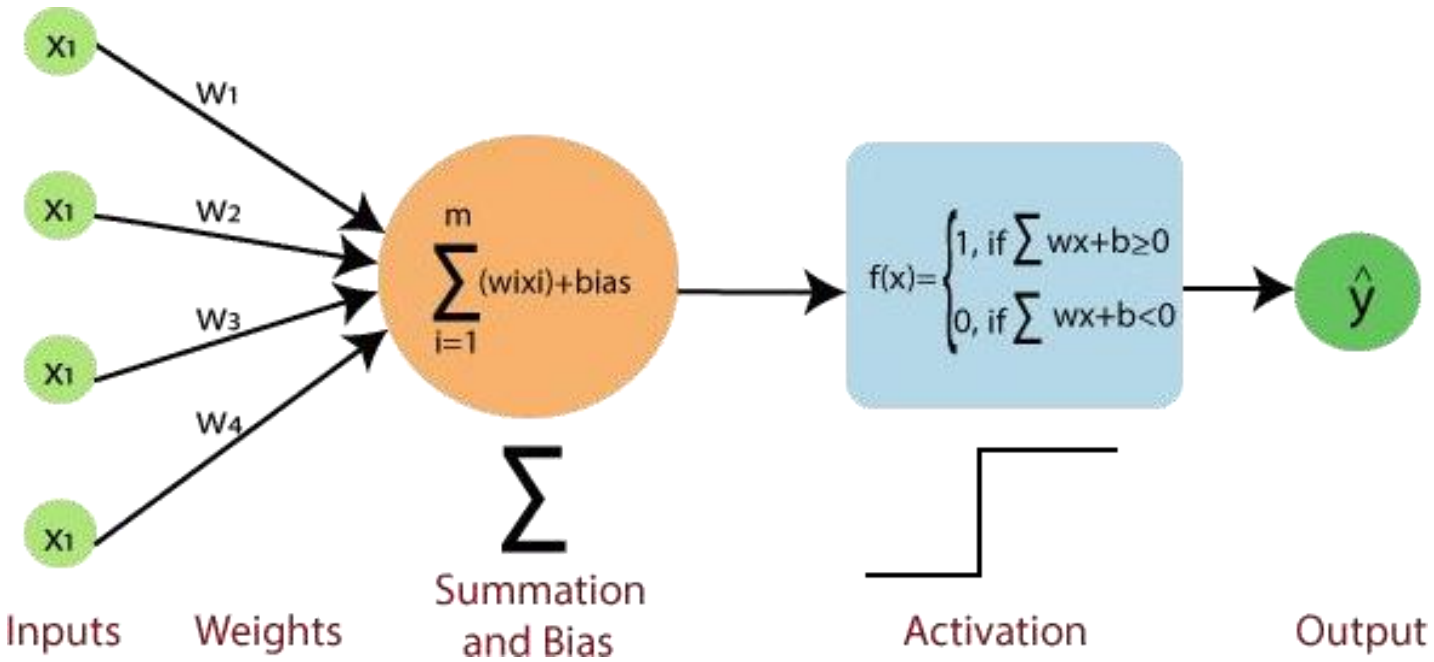
3. Ensemble Methods: Random Forest Classifier

```
# Import scikit-learn dataset library
from sklearn import datasets
# Load dataset
dat = datasets.load_iris()
X = dat.data
y = dat.target
# Split dataset into training set and test set 70% training and 30% test
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=100)
# training the model on training set
from sklearn.ensemble import RandomForestClassifier
# n_estimators : The number of trees in the forest.
# max_depth : The maximum depth of the tree.
# n_jobsint : The number of jobs to run in parallel
rf_clf  =  RandomForestClassifier(n_estimators=10)
rf_clf.fit(X_train,y_train)
# making predictions on the testing set
predicted = rf_clf.predict(X_test)
# comparing actual response values (y_test) with predicted response values (predicted)
from sklearn import metrics
print("Classification report : \n", rf_clf,"\n", metrics.classification_report(y_test, predicted))
disp = metrics.plot_confusion_matrix(rf_clf, X_test, y_test)
disp.figure_.suptitle("Confusion Matrix")
print("Confusion matrix: \n", disp.confusion_matrix)
```

Neural Networks

4. Neural Networks - Perceptron

- The inputs and output are numbers, and each input connection is associated with a weight.
- Compute the **weighted sum of its inputs** then applies an **activation function** to that sum and **outputs the result**.



4. Neural Networks - Perceptron

Input :$(\mathcal{D}, \mathbf{w}^{(0)})$.

Output :$\mathbf{w}$.

for each data point $\mathbf{x}^{(j)}, j = 1, \ldots n$, **do**

  if $\left(y^{(j)} = +1 \text{ and } f(\mathbf{x}^{(j)}) \leq 0\right)$ **then**

    $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \alpha\, \mathbf{x}^{(j)}$

  else **if** $\left(y^{(j)} = -1 \text{ and } f(\mathbf{x}^{(j)}) \geq 0\right)$ **then**

    $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha\, \mathbf{x}^{(j)}$

    else

    $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)}$.

    end

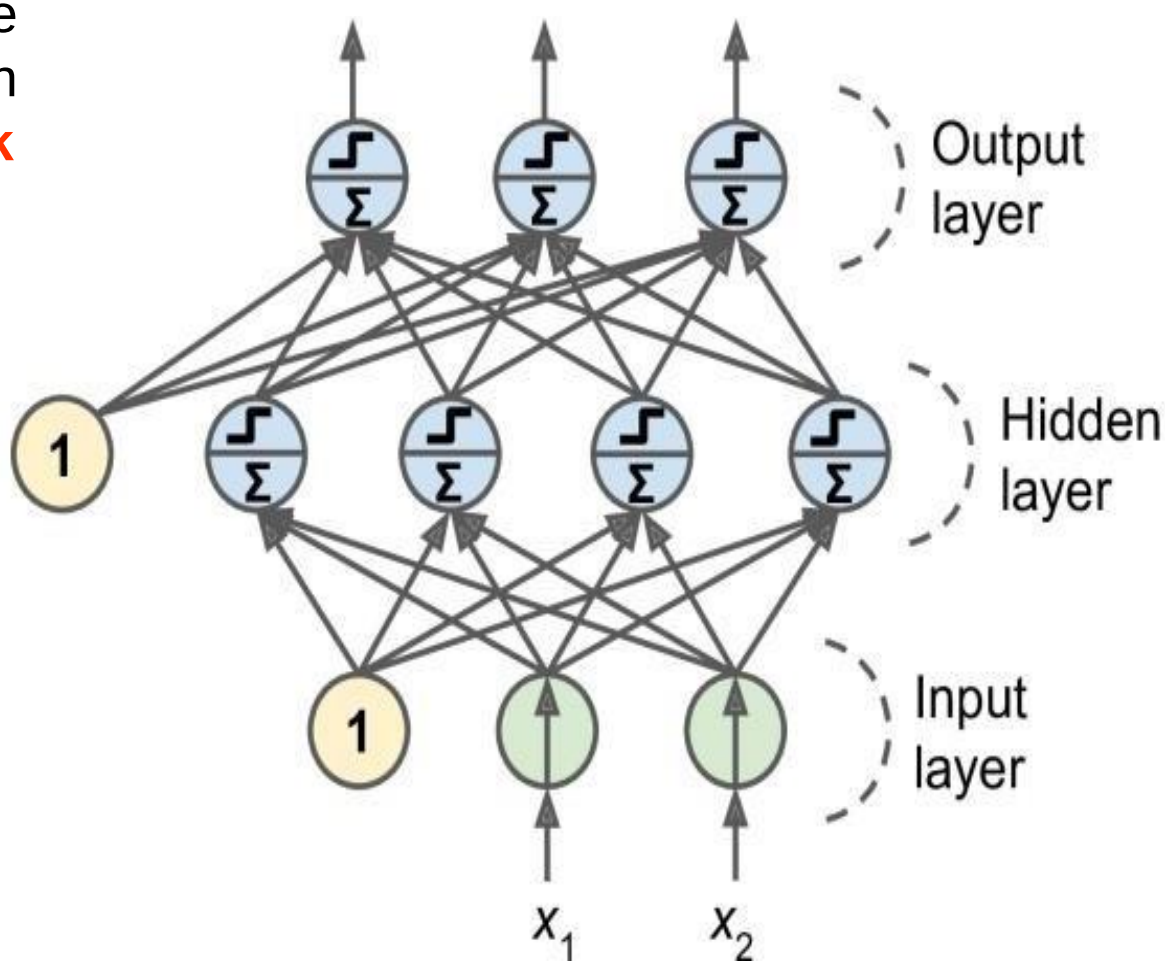
  end

end

4. Neural Networks - MLP

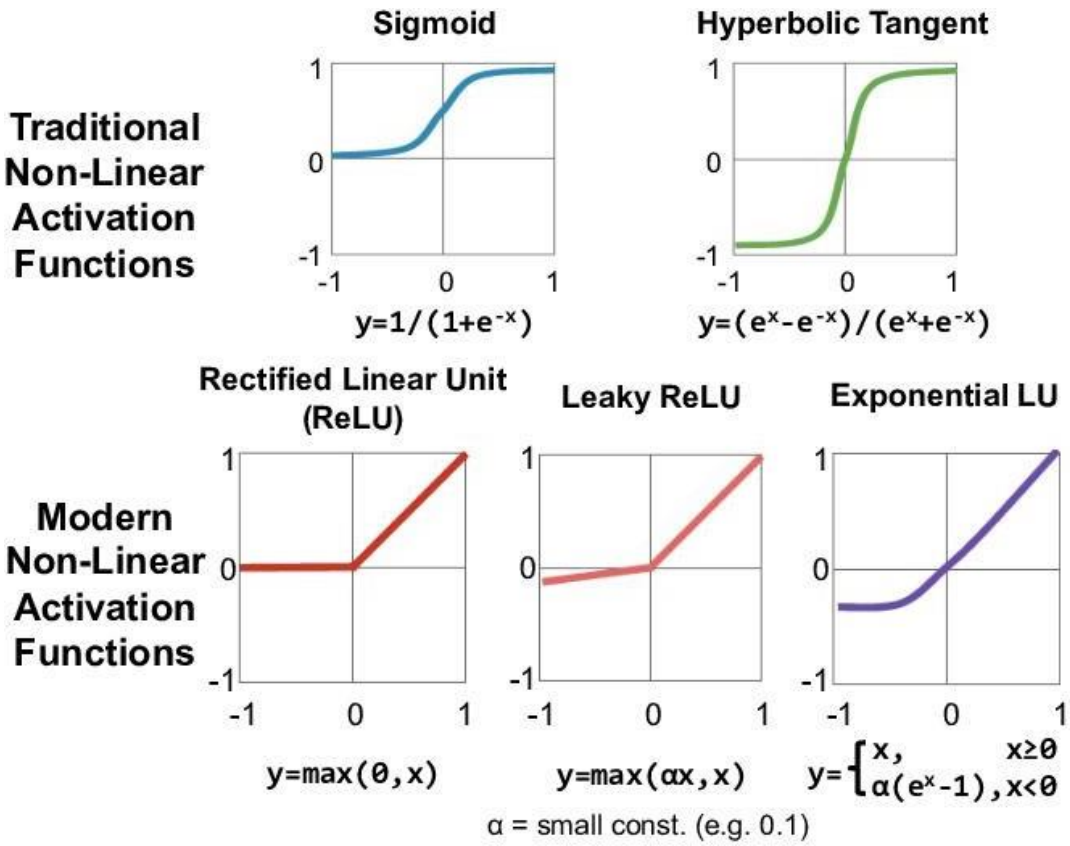
- The signal flows only in one direction (from the inputs to the outputs), so this architecture is an example of a **feedforward neural network (FNN)**.

The backpropagation consists of only two passes through the network **(one forward, one backward)**, the backpropagation algorithm is able to compute the gradient of the **network's error** with regard to every single model parameter. In other words, it can find out how each **connection weight** and each **bias** term should be tweaked in order to reduce the error.



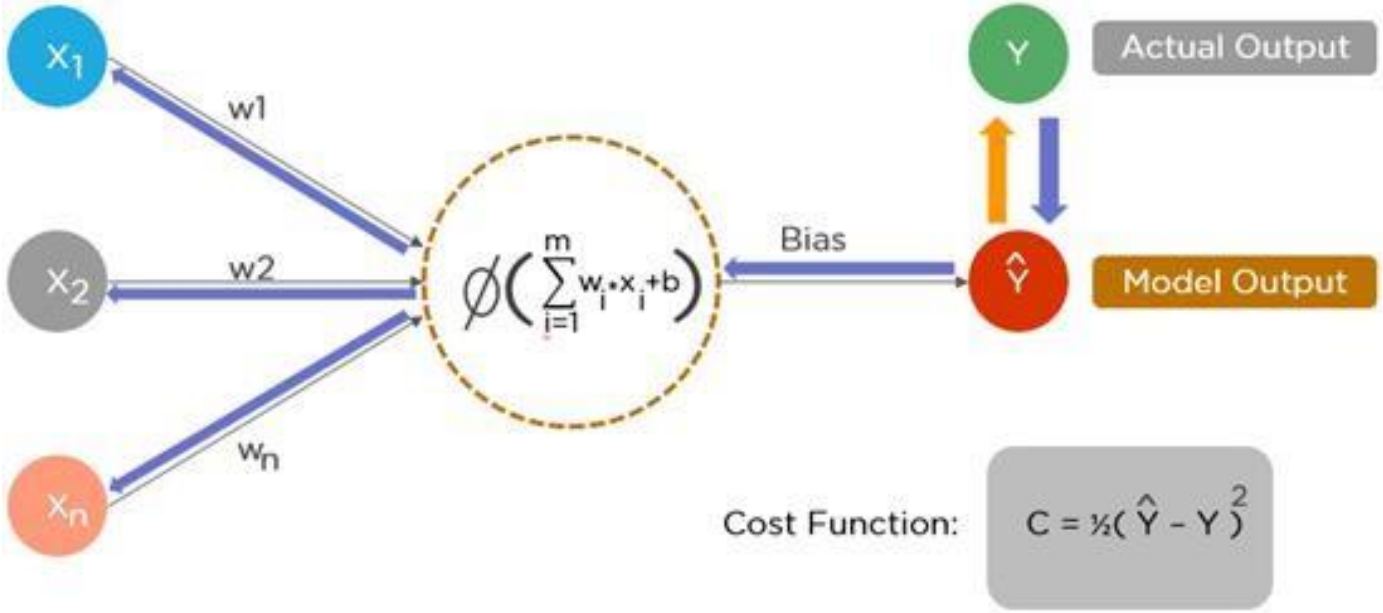
4. Neural Networks - MLP

1) Cost Function
2) Gradient Descent
3) Learning Rate
4) Backpropagation
5) Batch_size
6) Epochs



Neural Network vocabulary

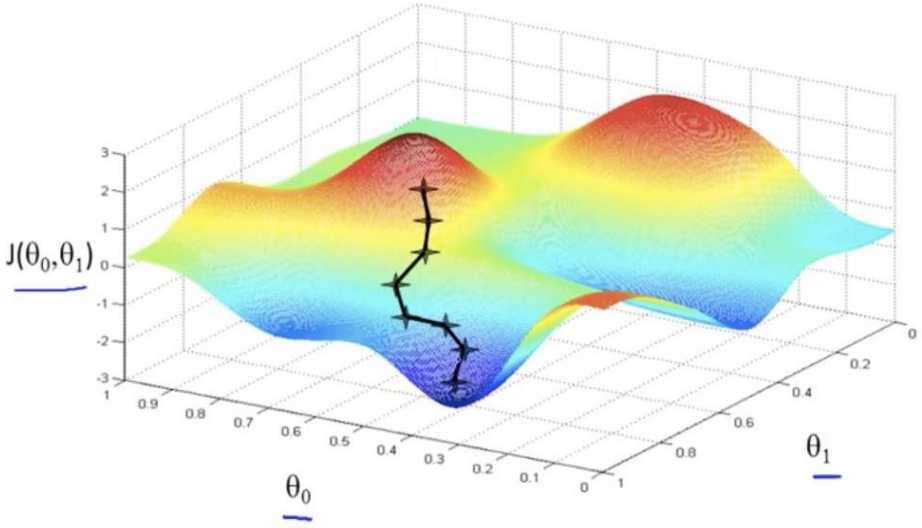
Cost Function: When we build a network, the network tries to predict the output as close as possible to the actual value. We measure this accuracy of the network using the cost/loss function. **The cost or loss function tries to penalize the network when it makes errors.**

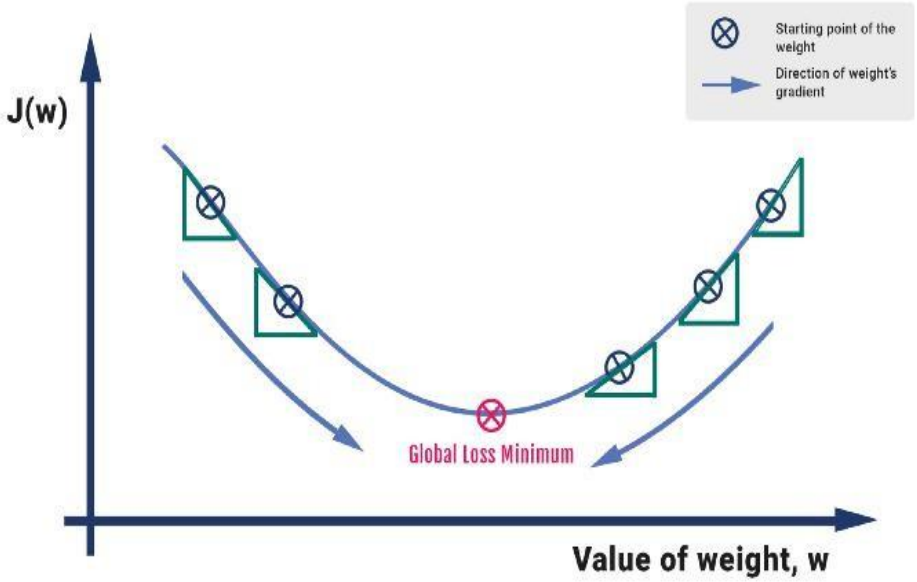


Neural Network vocabulary

Gradient descent: is an optimization algorithm used to minimize some function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient. In machine learning, we use gradient descent to **update the parameters of our model**. Parameters refer to **weights** in neural networks. (**solver in scikit-learn**)

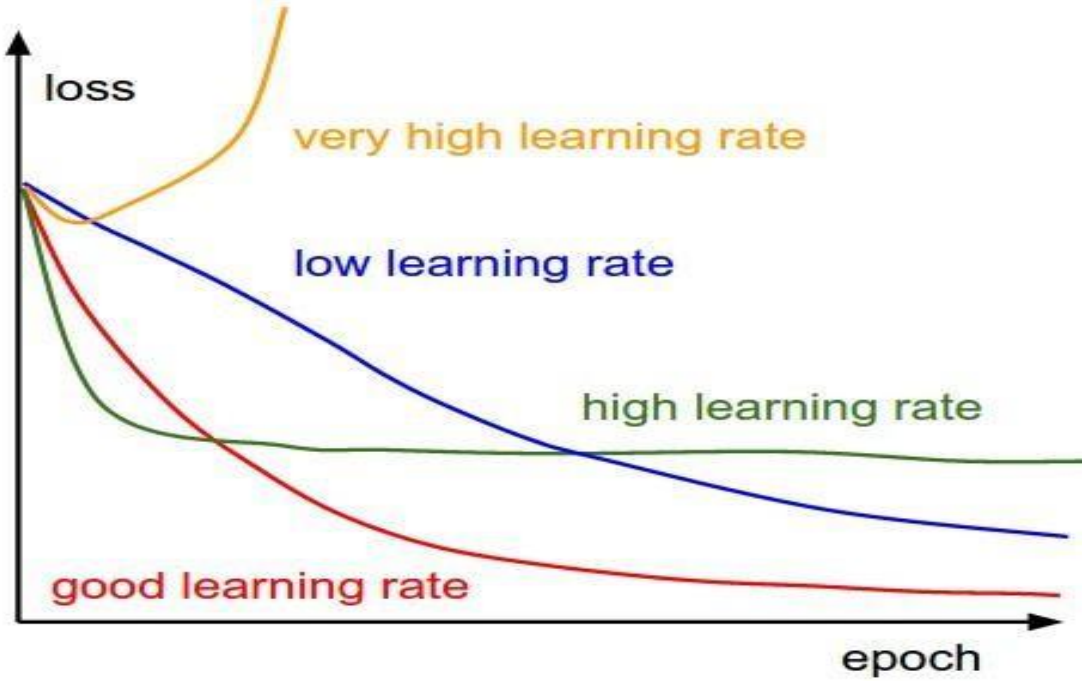
To put it simply, we use gradient descent to minimize the cost function, **J(w)**.





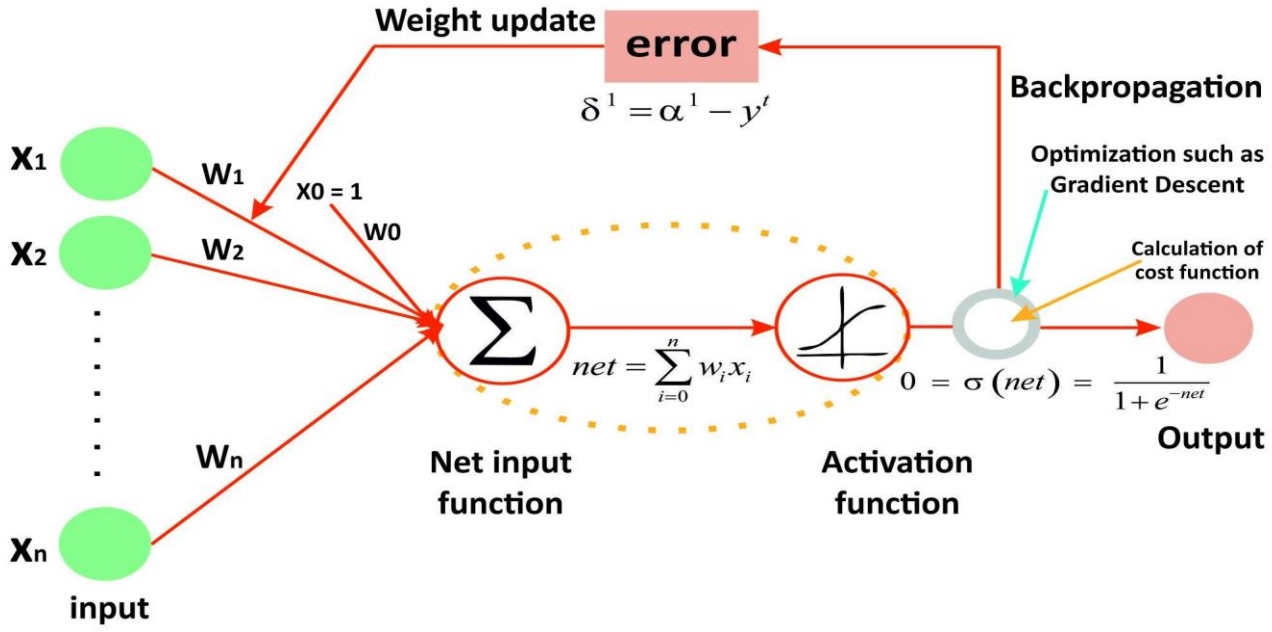
Neural Network vocabulary

Learning Rate: is defined as the amount of minimization in the cost function in each iteration. In simple terms, the rate at which we descend towards the minima of the cost function is the learning rate. We should choose the learning rate very carefully since it should neither be very large that the optimal solution is missed and nor should be very low that it takes forever for the network to converge.



Neural Network vocabulary

Backpropagation: after we calculated the error of the network, this error is then fed back to the network along with the gradient of the cost function to update the weights of the network. These weights are then updated so that the errors in the subsequent iterations is reduced. This updating of weights using the gradient of the cost function is known as backpropagation.



Neural Network vocabulary

Batches: while training a neural network, instead of sending the entire input in one go, we divide in input into **several chunks of equal size randomly**. Training the data on batches makes the model more generalized as compared to the model built when the entire data set is fed to the network in one go.

- **It requires less memory.**
- **Typically networks train faster with mini-batches.**

- **Batch Gradient Descent : Batch Size = Size of Training Set**
- **Stochastic Gradient Descent: Batch Size = 1**
- **Mini-Batch Gradient Descent: 1 < Batch Size < Size of Training Set**

Neural Network vocabulary

Epochs: an epoch is defined as **a single training iteration** of all batches in both **forward and backpropagation**. This means 1 epoch is a single forward and backward pass of the entire input data.

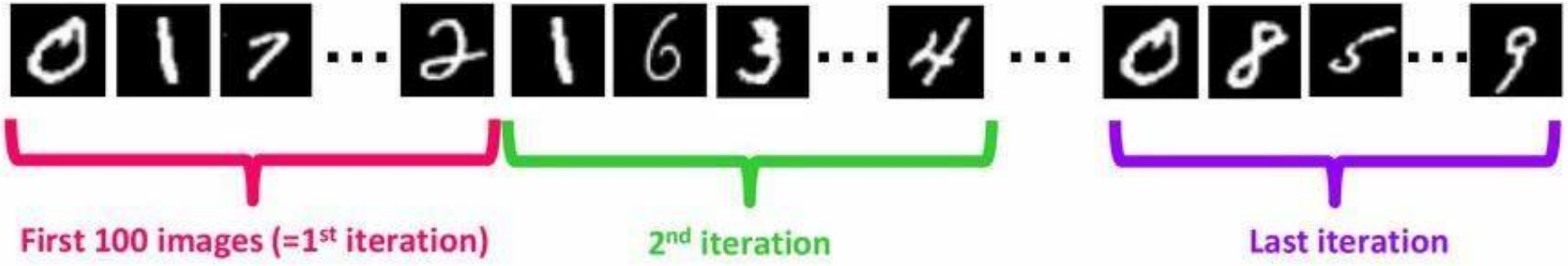
The number of epochs you would use to train your network can be chosen by you. It is highly likely that **more number of epochs would show higher accuracy** of the network, however, it would also take longer for the network to converge. Also, you must take care that if the number of epochs are too high, the network might be over-fit.

Neural Network vocabulary

Epoch / Iteration

Example: MNIST data

- number of training data: **N=55,000**
- Let's take batch size of **B=100**



- How many iteration in each epoch? **55000/100 = 550**

1 epoch = 550 iteration

4. Neural Networks - MLP

```
# Classification

# Feature Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_trainNN)

X_trainNN = scaler.transform(X_trainNN)
X_testNN = scaler.transform(X_testNN)

# Training and Predictions
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(128, 64,), batch_size=64,
                    solver="adam", activation="relu", max_iter=1000)

mlp.fit(X_trainNN, y_trainNN)

predicted = mlp.predict(X_testNN)
```

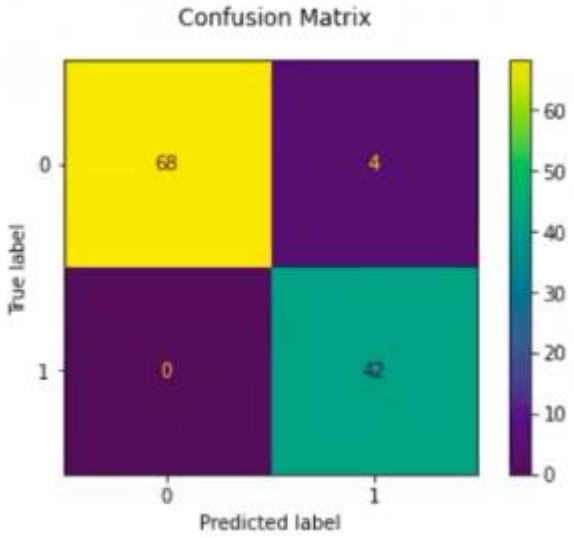
4. Neural Networks - MLP

```
# Classification
from sklearn import datasets, metrics
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# Feature Scaling
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)

X_trainNN = scaler.transform(X_train)
X_testNN = scaler.transform(X_test)


# Training and Predictions

from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(100, 100,), batch_size=128,
                    solver="lbfgs", alpha=0.1, activation="logistic",
                    max_iter=10000, random_state=42)
mlp.fit(X_trainNN, y_train)
```

4. Neural Networks - MLP

```
predicted = mlp.predict(X_testNN)

# comparing actual response values (y_test) with predicted response values (predicted)
print("Classification report : \n", metrics.classification_report(y_test, predicted))
disp = metrics.plot_confusion_matrix(mlp, X_testNN, y_test)
disp.figure_.suptitle("Confusion Matrix")
print("Confusion matrix: \n", disp.confusion_matrix)
```

```
Confusion matrix:
 [[68  4]
 [ 0 42]]
```



4. Neural Networks - MLP

```
# Regression

# Feature Scaling
from sklearn import preprocessing
scaler = preprocessing.StandardScaler()
scaler.fit(X_trainNN)

X_trainNN = scaler.transform(X_trainNN)
X_testNN = scaler.transform(X_testNN)

# Training and Predictions
from sklearn.neural_network import MLPRegressor
mlpR = MLPRegressor(hidden_layer_sizes=(128, 64,), batch_size=64,
                    solver="adam", activation="relu", max_iter=1000)

mlpR.fit(X_trainNN, y_trainNN)

predicted = mlpR.predict(X_testNN)
```